Can you help me to provide a dession tree by which can help me to decide either use monolithic or microservices archtech in my project.
I now veries project to project but I want a common and detailed desion tree which may help me to decide.

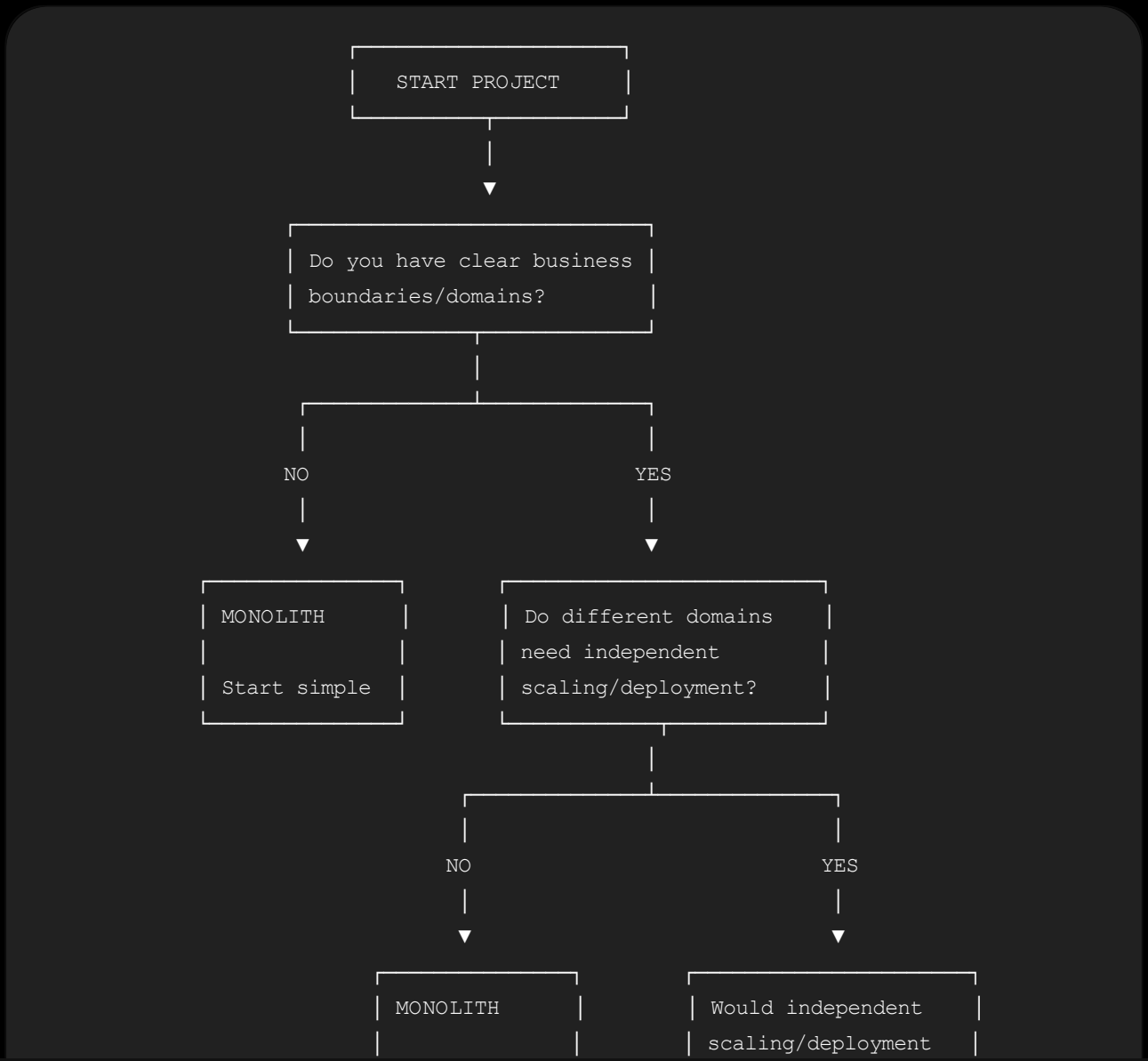
Yes. And I would actually recommend **not making the decision simply as "small project = monolith, big project = microservices."** That rule is too simplistic.

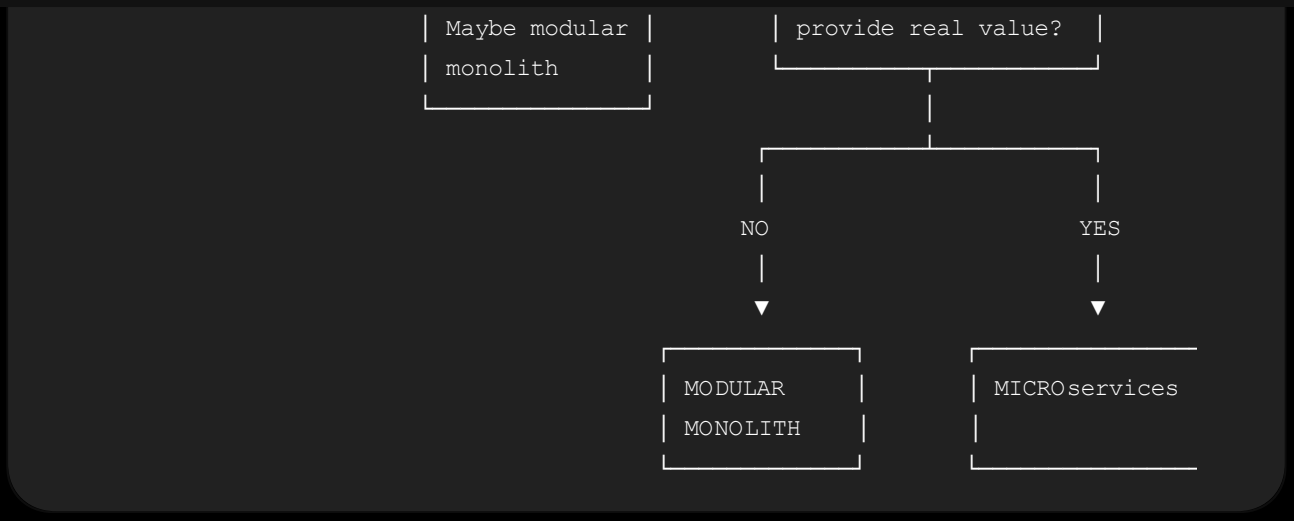
A better approach is to evaluate the project across **business boundaries, scaling, team structure, deployment, data, reliability, and operational complexity**.

Below is a reusable decision tree you can apply to almost any project.

Monolith vs Microservices Decision Tree

Start here:





But this is only the **first layer**. In real projects, I recommend going through the following decision framework.

---

1. Do you actually have separate business domains?

Ask:

> "Can I identify genuinely independent business capabilities?"

For example, an e-commerce platform might have:

```
Users
Orders
Payments
Inventory
Shipping
Notifications
Analytics
```

But simply having different modules does **not** mean you need microservices.

Ask:

- Does each domain have its own business rules?
- Can it evolve independently?
- Does it have its own data?
- Could it potentially be owned by a separate team?
- Does it have a different scaling profile?
- Could it be deployed independently?

Decision

```
No clear boundaries
       ↓
   MONOLITH

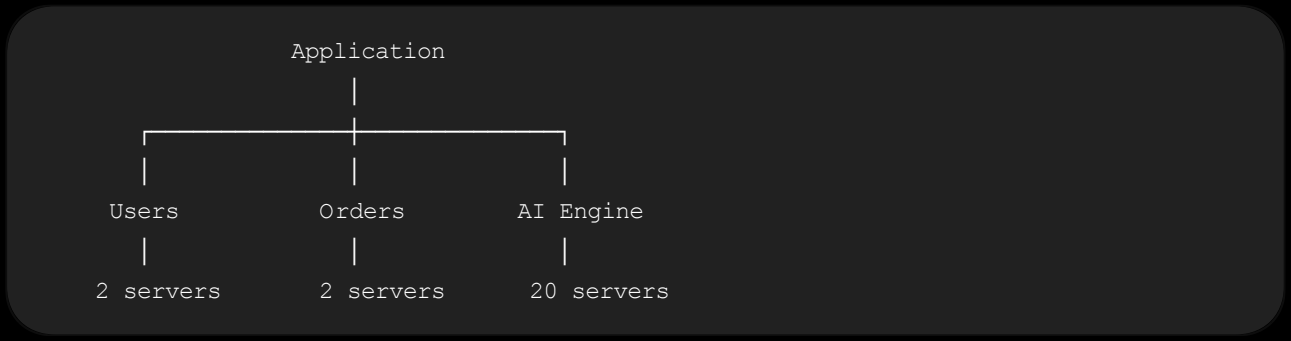
Clear boundaries
       ↓
Continue
```

---

2. Do different components need independent scaling?

This is one of the strongest reasons for microservices.

Imagine:



If the AI engine needs 20 servers but the rest of your application only needs 2, separating it can make sense.

Ask:

> "Do different parts of the system have significantly different resource requirements?"

Examples:

Strong microservice candidate

```
Web API
   |
   ├── Users       → low CPU
   ├── Orders      → medium CPU
   ├── AI          → high CPU/GPU
   ├── Video       → high CPU
   └── Analytics   → high memory
```

Decision

```
Similar scaling requirements
          ↓
```

```
       MONOLITH

Very different scaling requirements
          ↓
Continue evaluating
```

---

3. Do you need independent deployments?

Ask:

> "Do I frequently need to deploy one part without deploying the whole application?"

For example:

```
Payment Service
       ↓
Deploy every few days

User Service
       ↓
Deploy once/month

AI Service
       ↓
Deploy multiple times/day
```

If independent deployment is genuinely valuable:

```
YES → Microservices become more attractive
NO  → Monolith remains attractive
```

Important

Don't create microservices just because:

> "It would be nice to deploy independently."

There should be an actual business/engineering benefit.

---

4. How large is the development team?

This is often overlooked.

1-5 developers

Usually:

```
MONOLITH
```

Microservices create additional work:

```
Service management
Networking
Deployment
Monitoring
Logging
Authentication
API contracts
Retries
Queues
Distributed transactions
Service discovery
```

Your small team can end up spending more time managing infrastructure than building features.

5-15 developers

Could be:

```
MODULAR MONOLITH
        OR
HYBRID
```

Start with a modular monolith and extract services where justified.

15-50+ developers

Microservices become more reasonable, particularly when teams can own domains.

For example:

```
Team A → Payments
Team B → Orders
Team C → Users
Team D → AI
Team E → Notifications
```

Rule

> **Microservices are often as much an organizational architecture as a technical architecture.**

---

5. Do different teams need to work independently?

Ask:

> "Will multiple teams frequently modify the same codebase?"

If:

```
10 developers
       ↓
same repository
       ↓
same deployment
       ↓
frequent conflicts
```

microservices can help.

But if:

```
5 developers
       ↓
same product
       ↓
same business domain
```

a monolith is usually simpler.

---

6. Do services require different technologies?

Suppose:

```
Main application → Laravel

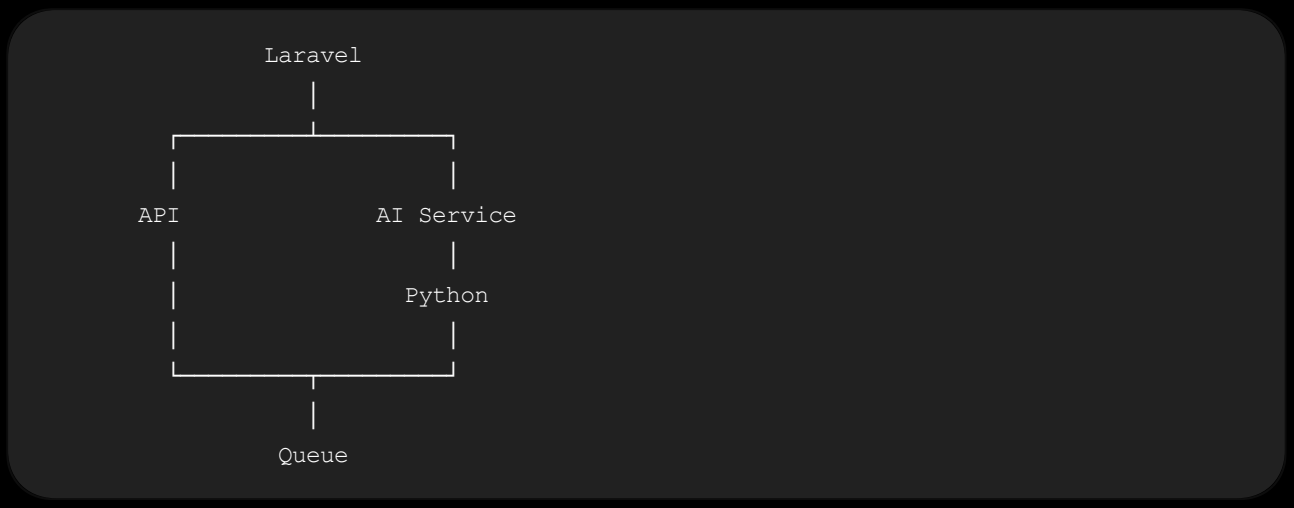
AI → Python

Realtime → Node.js

Data processing → Python
```

This can be a legitimate reason for service separation.

For example:



But don't fall into:

> "Python is better for AI, therefore everything should become microservices."

You can have a **modular monolith + one Python service**.

That's often a much better architecture.

---

7. Do you need strong isolation?

Some domains may need isolation because a failure should not bring down the whole system.

Example:



If the trading component crashes, you may want:

```
Trading → isolated
Users → still working
Reporting → still working
```

This strongly favors service isolation.

---

8. Do you need independent security boundaries?

Ask whether a component has substantially different:

- security requirements
- credentials
- network access
- compliance requirements
- data sensitivity

For example:

```
Public API
     |
     ├── User Service
     |
     ├── Payment Service
     |
     └── Sensitive Financial Service
```

If the financial component requires strong isolation, a separate service can be justified.

---

9. Is the system transaction-heavy?

This is a major consideration.

Imagine:

```
Create Order
     ↓
Reserve Inventory
     ↓
Charge Payment
     ↓
Create Shipment
     ↓
Update Loyalty Points
```

Inside a monolith, this can potentially be handled through a database transaction.

With microservices:

```
Order Service
      ↓
Inventory Service
      ↓
Payment Service
      ↓
Shipping Service
```

Now you have:

- distributed transactions
- retries
- duplicate messages
- eventual consistency
- failure handling
- idempotency

This dramatically increases complexity.

Therefore:

```
Strong ACID transactions across many modules
             ↓
          MONOLITH
```

unless there's a compelling reason otherwise.

---

10. Can the domains tolerate eventual consistency?

Microservices often require:

```
Service A
   ↓
Message/Event
   ↓
Service B
   ↓
Message/Event
   ↓
Service C
```

Data may temporarily be:

```
Order = PAID
Inventory = processing
```

Ask:

> "Is temporary inconsistency acceptable?"

If NO

Prefer:

```
MONOLITH
```

If YES

Microservices become more viable.

---

11. Does the system need high availability?

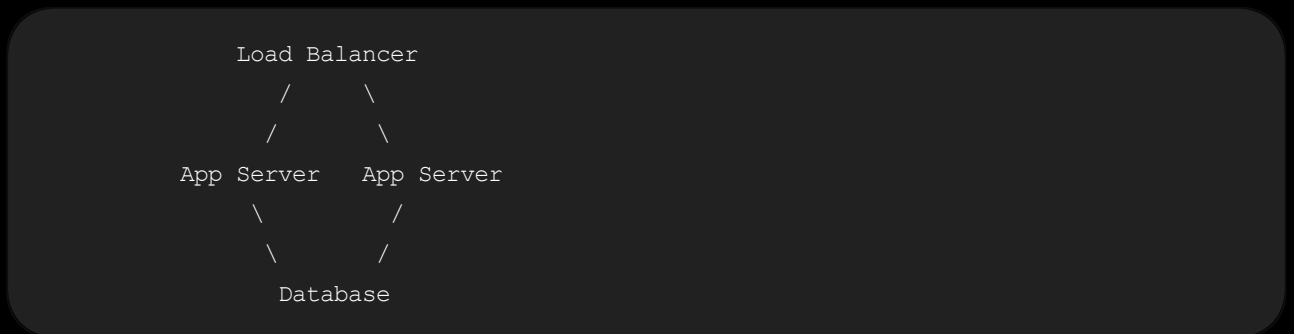
This requires careful thinking.

People often say:

> "We need high availability, therefore microservices."

Not necessarily.

A well-designed monolith can be highly available:



You don't automatically get HA by using microservices.

Microservices can actually introduce **more failure points**.

---

12. Does the system need massive scale?

Ask:

> "What is the expected scale?"

For example:

```
10K users
     ↓
MONOLITH is probably enough


1M users
     ↓
Could still be MONOLITH


100M users
     ↓
Microservices become more attractive
```

But **user count alone isn't enough**.

A better metric is:

```
Requests/sec
Database load
CPU usage
Memory
Background jobs
Data volume
Traffic patterns
Scaling differences
```

A 1M-user application can still be perfectly fine as a modular monolith.

---

13. Do you have DevOps maturity?

This is extremely important.

Ask:

Do you already have:

- Docker?
- CI/CD?
- Kubernetes/ECS/etc.?
- Centralized logging?
- Monitoring?
- Distributed tracing?
- Alerting?

- Secrets management?
- Automated testing?
- Infrastructure as code?

If:

```
DevOps maturity = LOW
```

then:

```
MONOLITH
```

is usually the safer choice.

If:

```
DevOps maturity = HIGH
```

microservices become easier to operate.

---

14. Do you have observability?

With monolith:

```
Request
   ↓
Application
   ↓
Database
```

With microservices:

```
Request
  ↓
API Gateway
  ↓
Service A
  ↓
Service B
  ↓
Queue
  ↓
Service C
  ↓
Database
```

When something fails, you need to answer:

> Where did it fail?

You may need:

```
Centralized logs
Metrics
Distributed tracing
Correlation IDs
Health checks
Alerts
```

If you're not prepared for this, don't rush into microservices.

---

15. Does the project need independent database ownership?

A proper microservice architecture generally moves toward:

```
User Service
    ↓
User DB

Order Service
    ↓
Order DB

Payment Service
    ↓
Payment DB
```

rather than:

```
User Service ──┐
Order Service ─┼──→ SAME DB
Payment Service┘
```

The second approach often creates a **distributed monolith**.

Ask:

> "Can each service own its data?"

If the answer is no, microservices may not provide much value.

16. How expensive would failure be?

Consider:

```
Failure impact
      ↓
   Very high
      ↓
Need isolation?
```

Examples:

- financial transactions
- trading
- payment processing
- critical healthcare systems
- large-scale communication
- infrastructure services

These can justify stronger service boundaries.

But again, **criticality does not automatically mean microservices**.

---

17. Is the architecture likely to change significantly?

For a new product, you often don't know:

```
Which feature will become popular?
Which domain will scale?
Which business model will survive?
```

Therefore:

Early-stage product

Usually:

```
MODULAR MONOLITH
```

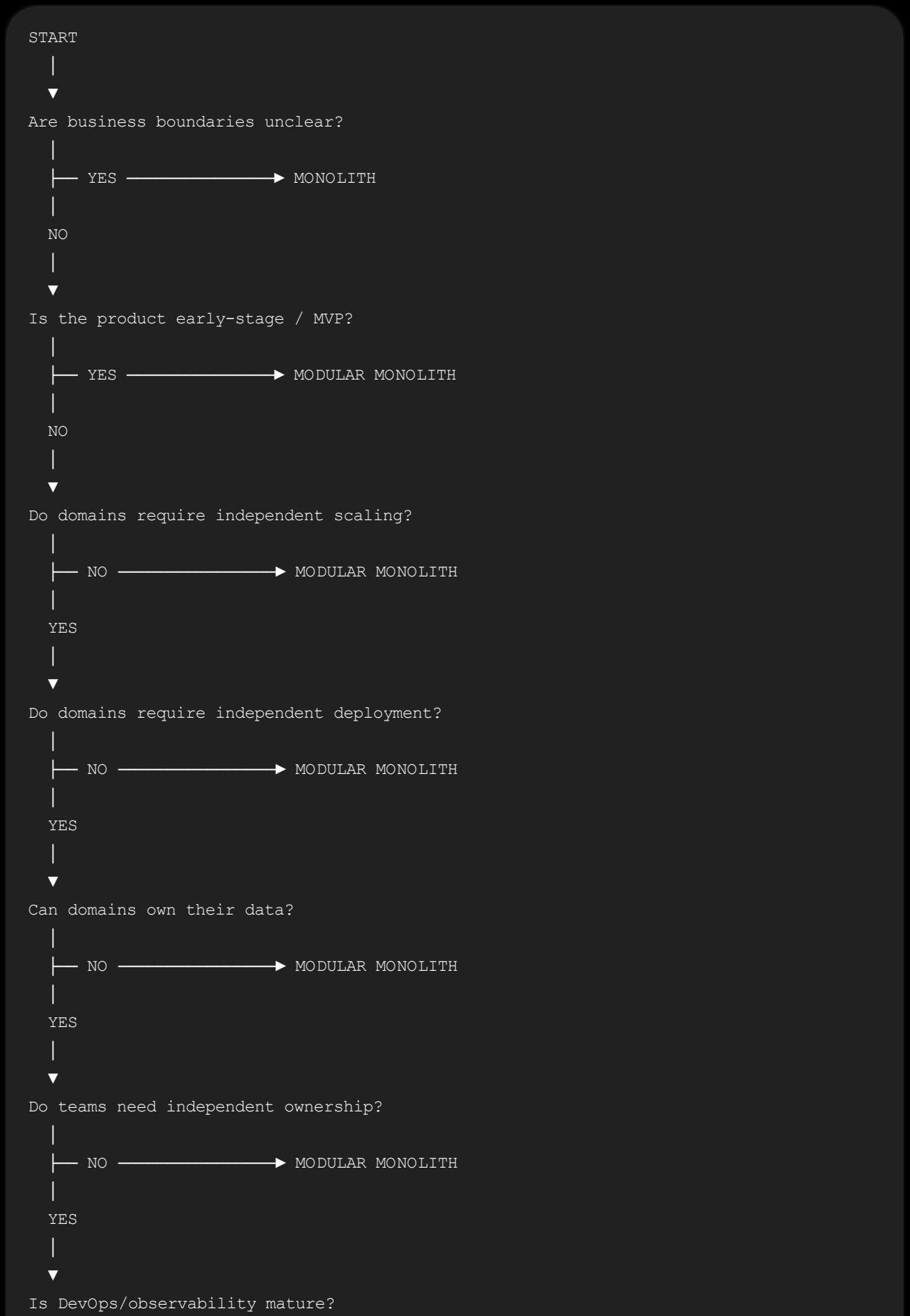
rather than:

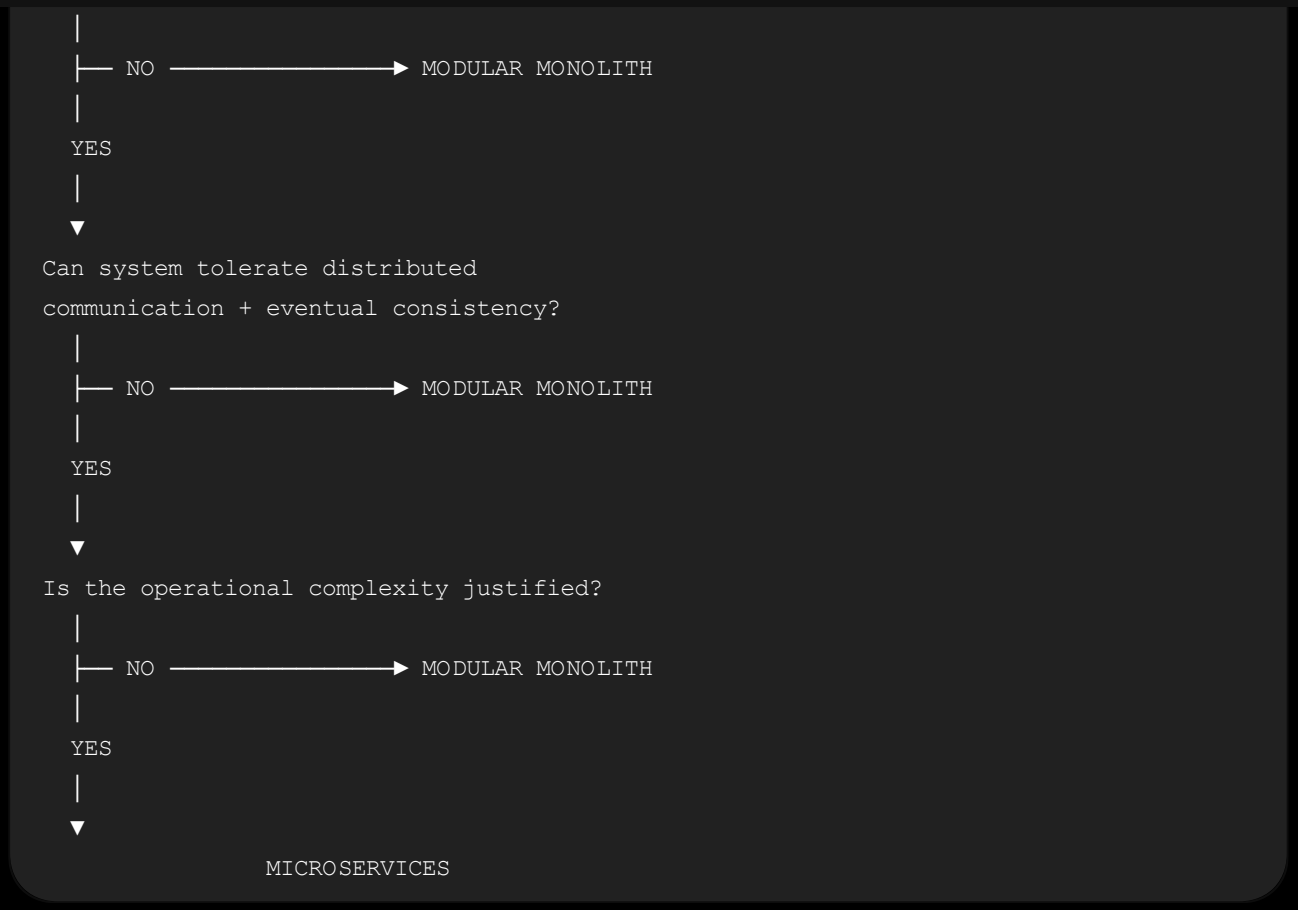
```
20 microservices
```

You can extract services later.

The architecture decision tree I recommend

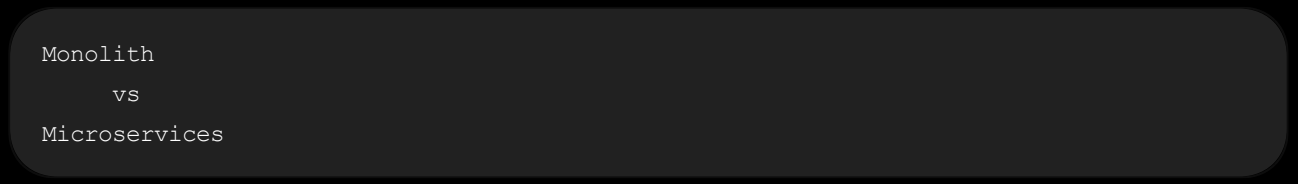
Here is the version I would actually use in your projects:



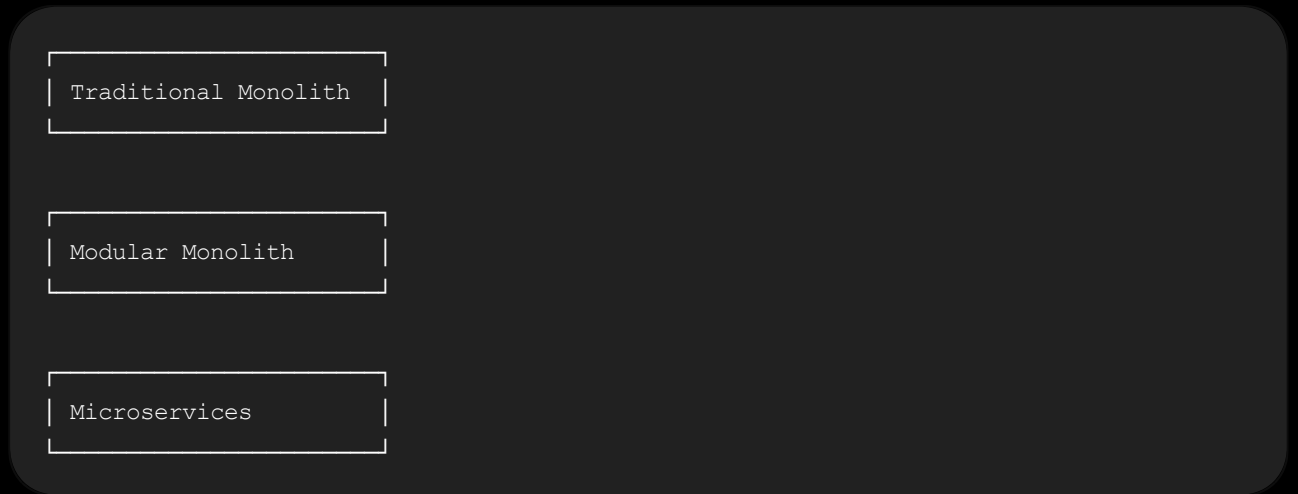


But there is a third option you should use a lot

I wouldn't make your decision binary:



Use **three choices**:



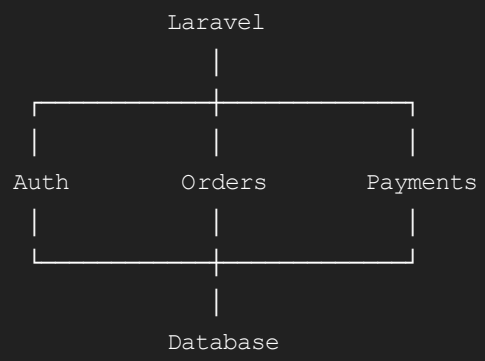
And for many modern projects, **Modular Monolith should be your default starting point**.

What does Modular Monolith mean?

For example, in Laravel:

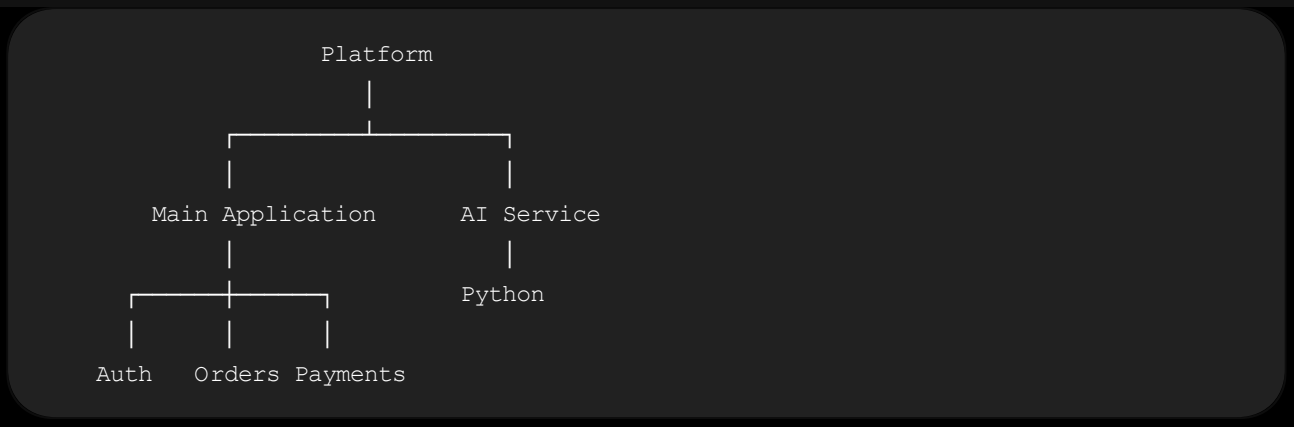
```
Laravel Application
│
├── Auth
│   ├── Controllers
│   ├── Services
│   ├── Models
│   └── Policies
│
├── Users
│   ├── Controllers
│   ├── Services
│   ├── Models
│   └── Policies
│
├── Orders
│   ├── Controllers
│   ├── Services
│   ├── Models
│   └── Policies
│
├── Payments
│   ├── Controllers
│   ├── Services
│   └── Models
│
└── Notifications
    ├── Services
    └── Jobs
```

Everything is deployed together:



But the boundaries are clean.

Later:



Then perhaps:



You **extract services when there is a reason**, rather than designing 15 services on day one.

A practical scoring system

You can make your decision even more systematic.

Score each factor from **0 to 3**:

Factor	0	1	2	3
Clear domain boundaries	None	Weak	Good	Very strong
Independent scaling	None	Small	Significant	Extreme
Independent deployment	None	Rare	Useful	Critical
Team independence	1 team	Some separation	Multiple teams	Fully independent

Factor	0	1	2	3
Technology differences	Same	Minor	Some	Major
Fault isolation	Low need	Some	Important	Critical
Data ownership	Shared	Mostly shared	Mostly independent	Fully independent
Expected scale	Small	Medium	Large	Massive
DevOps maturity	Low	Basic	Good	Excellent
Observability	Low	Basic	Good	Excellent
Eventual consistency tolerance	No	Limited	Mostly	Yes
Operational resources	Low	Limited	Good	Excellent

Then:

0-10

Traditional Monolith

11-20

Modular Monolith

21-28

Modular Monolith + selective services

29+

Microservices may be justified

But don't treat this as mathematical truth. The **questions matter more than the score**.

My rule of thumb for your projects

Given the type of applications you work on, I would use this:

MVP / startup

```
Modular Monolith
+
Queue
+
Cache
+
Object Storage
```

Normal SaaS

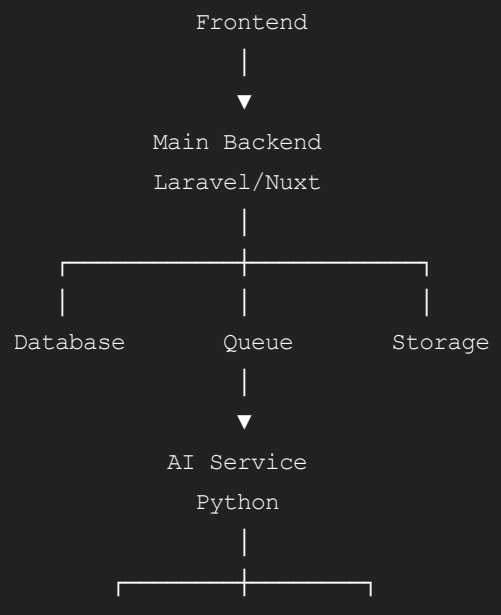
```
Modular Monolith
+
Background Workers
+
Queue
+
Cache
```

Then extract only the services that need it.

AI application

Don't automatically make the entire application microservices.

I would often do:



This is often much more sensible than turning every feature into a microservice.

FinTech / Trading

Here I would evaluate much more carefully:

```
Main Platform
     |
     ├── Identity
     ├── Users
     ├── CRM
     ├── Reporting
     |
     ├── Trading Engine ──→ isolated
     ├── Payment ─────────→ isolated
     └── Notifications
```

The critical components may deserve independent services even if the rest remains modular.

The most important principle

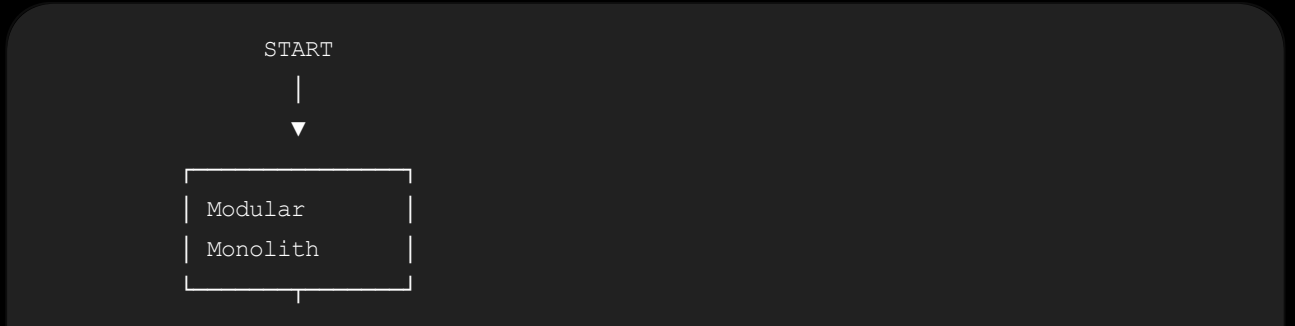
I would remember this one sentence:

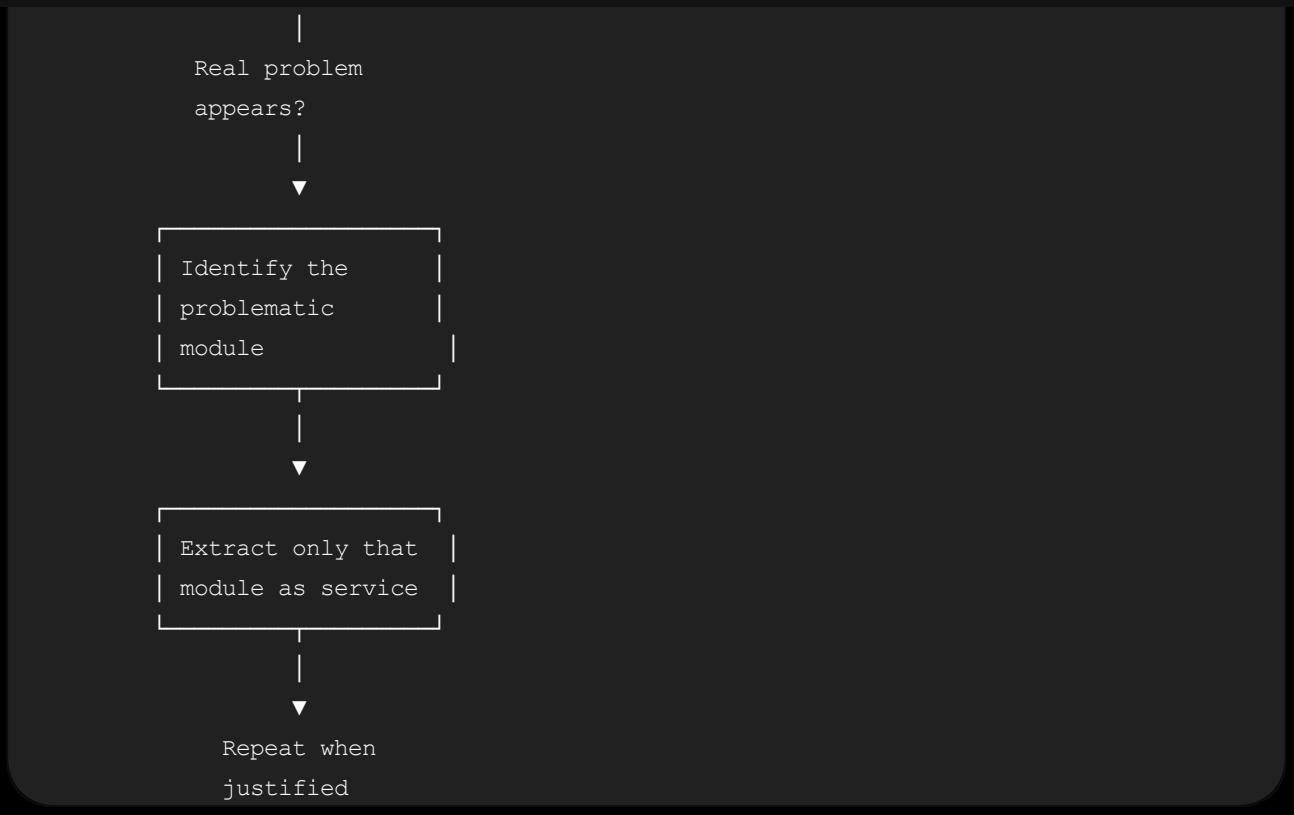
> **Don't choose microservices because your application is large. Choose microservices when the benefits of independent boundaries, scaling, deployment, ownership, and fault isolation outweigh the complexity of distributed systems.**

And the opposite:

> **Don't choose a monolith because your application is small. Choose it when simplicity, transactional consistency, development speed, and operational efficiency provide more value.**

For most projects, my default progression would be:





This approach gives you a **migration path from monolith → microservices**, instead of forcing you to make a potentially expensive architecture decision before you understand the system.

I can also create an image explaining this decision tree visually.

Create an image explaining this decision tree visually



For practice I want a project from which i can get high level experience of all the architect like monolithic, modular monolith, hybrid and microservices.
I am thinking to first create a monolithic then convert that into modular monolithic and after that hybrid and at the last microservices.
Is this a right path or should I start with modular monolithic then convert it in micro services.
Give me the honest and best reply. You can disagree with me anytime.
I want a good practice project not simple a ecomrence. It

Show more

Worked for 19s

Yes. **For your specific goal, I disagree with the simpler recommendation of "start with a modular monolith and then move to microservices."**

If your objective is to **learn architecture deeply**, not just build a production project, then I think your original idea is better:

> **Monolith → Modular Monolith → Hybrid → Microservices**

But there is one important correction:

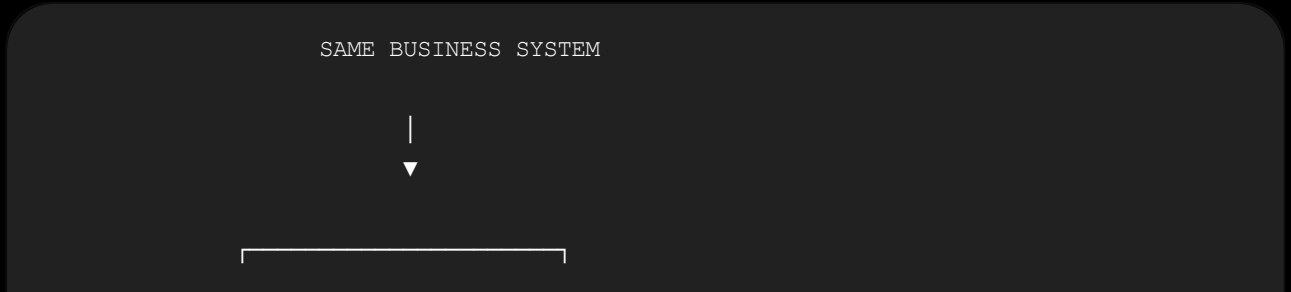
> Don't build a bad monolith and then spend the week fixing bad code. Build a **clean but intentionally non-modular monolith**, then progressively introduce architectural boundaries.

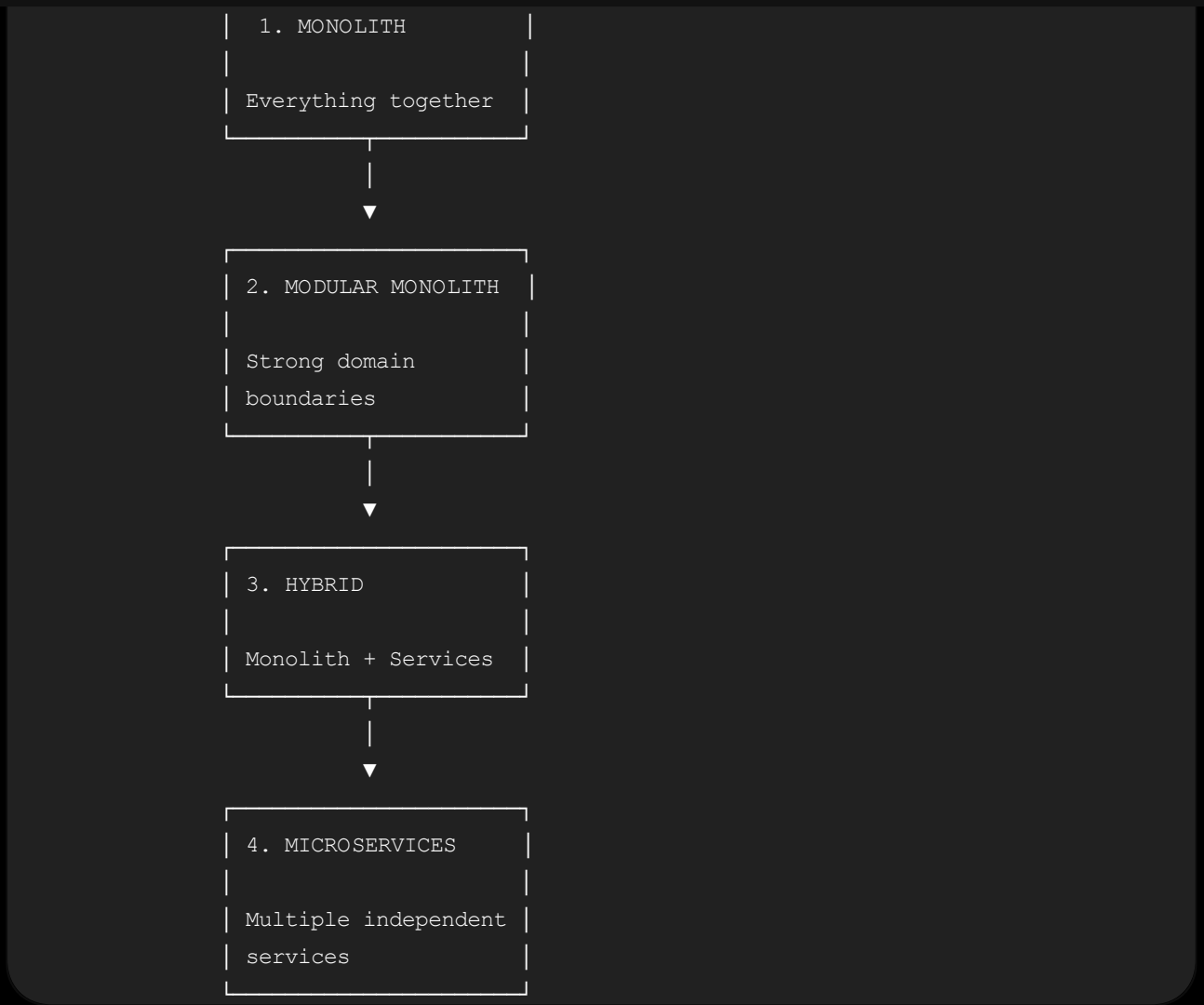
That will teach you *why* each architecture exists and what problems it solves.

---

1. What I recommend for you

I would build **one serious project** and evolve the same codebase four times.





This gives you something very valuable for interviews:

> **"I didn't just build microservices. I experienced why we would extract a service from a monolith, what problems appeared during extraction, and how we solved them."**

That is much stronger architectural knowledge.

---

2. The project I recommend

I don't want you to build another e-commerce system.

Considering your background in FinTech, I would build:

FinPay - Payment & Risk Processing Platform

Not a real-money system. Everything is simulated.

Think of it as a small version of a payment processing platform.

It has enough complexity to give you practice with:

- architecture
- transactions
- concurrency
- queues
- events
- caching
- authentication
- authorization
- idempotency
- retries
- failure handling
- distributed systems
- service communication
- database design
- background workers
- observability
- eventual consistency
- API design
- Docker
- CI/CD concepts

And importantly, it naturally supports your four architecture stages.

---

3. What should the system do?

Imagine merchants use your platform.



Example:

```
POST /payments

{
    "merchant_id": 123,
    "customer_id": 456,
    "amount": 250.00,
    "currency": "USD"
}
```

System:

```
Payment Request
      |
      ▼
Validate
      |
      ▼
Idempotency Check
      |
      ▼
Risk Analysis
      |
      ▼
Process Payment
      |
      ▼
Ledger Entry
      |
      ▼
Publish Event
      |
      ├──► Notification
      |
      └──► Reporting
```

This is **far more useful architecturally** than CRUD.

---

4. Core domains

I'd define approximately these domains:

```
FinPay
|
├── Identity
|
```

```
├── Merchants
│
├── Customers
│
├── Payments
│
├── Risk
│
├── Ledger
│
├── Notifications
│
└── Reporting
```

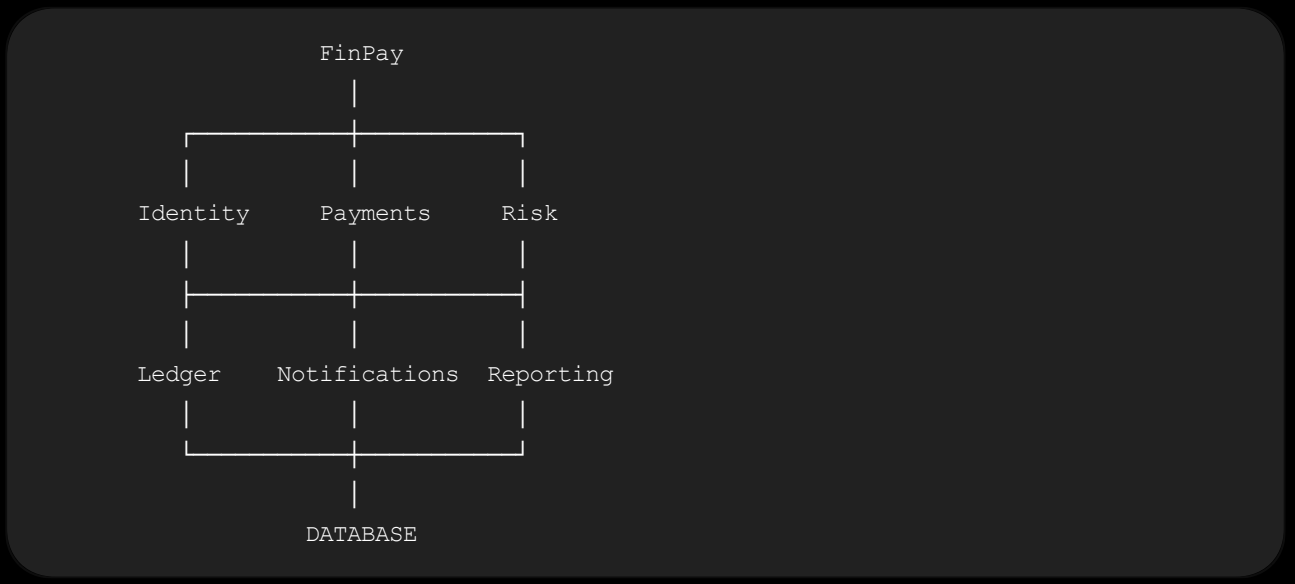
But we won't necessarily make each one a microservice.

That's actually the point of the exercise.

---

5. Stage 1: Traditional Monolith

First build:



One Laravel application.

One database.

One deployment.

One queue.

One codebase.

Something like:

```
app/
 ├── Models/
 ├── Controllers/
 ├── Services/
 ├── Jobs/
 ├── Events/
 └── ...
```

But don't make it spaghetti.

You can still use:

- service classes
- repositories where useful
- policies
- jobs
- events

The key is that the domains aren't strongly isolated yet.

---

6. What you should learn from Stage 1

Don't just build it.

After finishing it, deliberately ask:

Problem 1

What happens if Risk becomes CPU intensive?

```
Payment
   ↓
Risk calculation
   ↓
CPU 100%
   ↓
Entire application suffers
```

Problem 2

What if Reporting becomes very expensive?

Problem 3

What if Notifications fail?

Problem 4

What if Risk needs Python?

Problem 5

What if Reporting needs a different database?

Problem 6

What if Payments need 10 servers but Customers need only 2?

Now you have **real reasons** to change architecture.

---

7. Stage 2: Modular Monolith

Now refactor the same application.

```
Laravel
│
├── Identity
│
├── Merchant
│
├── Customer
│
├── Payment
│
├── Risk
│
├── Ledger
│
├── Notification
│
└── Reporting
```

Each module should have clear boundaries.

For example:

```
Payment
├── Controllers
├── Services
├── Models
├── Events
├── Jobs
└── Policies
```

And:

```
Risk
├── Services
├── Models
├── Events
└── Jobs
```

Now you should enforce rules such as:

> Payment should not directly manipulate Risk's internal models.

Instead:

```
Payment
   |
   ▼
Risk Interface
   |
   ▼
Risk Module
```

This is extremely important.

You're creating **seams for future extraction**.

---

8. Now introduce events

For example:

```
PaymentCreated
      |
      ├──────► Risk Analysis
      |
      ├──────► Notification
      |
      └──────► Reporting
```

Use a queue.

Now you'll start learning:

```
Synchronous
vs
Asynchronous
```

and:

```
Command
vs
Event
```

and:

```
Request/Response
vs
Event-driven communication
```

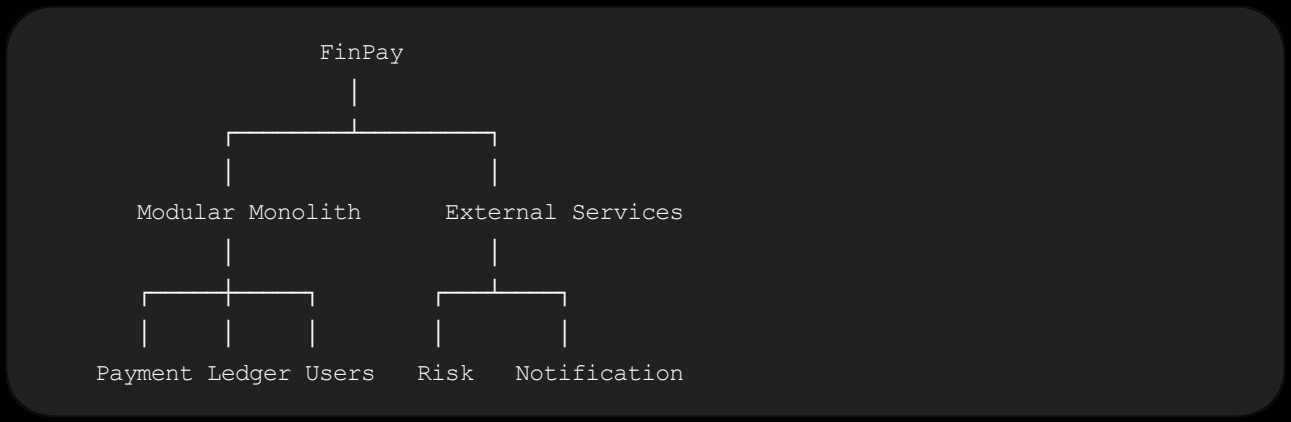
These are much more valuable architectural skills than simply knowing "microservices."

---

9. Stage 3: Hybrid Architecture

This is where the project becomes really interesting.

Keep your core application as a modular monolith:



I'd extract **Risk first**.

Why?

Because it gives you a legitimate reason.

Suppose Risk requires Python:

```
Laravel
   |
   | HTTP / REST
   ▼
Python Risk Service
```

Now you have:

```
Laravel
   |
```

```
   ├── Identity
   ├── Merchants
   ├── Customers
   ├── Payments
   ├── Ledger
   └── Reporting
           |
           ▼
       Python Risk
```

Then extract Notifications:

```
Payment
   |
   ▼
Event / Queue
   |
   ▼
Notification Service
```

Now you have your **hybrid architecture**.

---

10. This stage will teach you something extremely important

You'll suddenly encounter problems that didn't exist in the monolith.

For example:

```
Laravel
   |
   | HTTP
   ▼
Risk Service
```

What if Risk is down?

```
Payment
   |
   ▼
Risk
   |
   X
```

Now you need:

- timeout

- retry
- fallback
- circuit breaker concept
- idempotency
- error handling

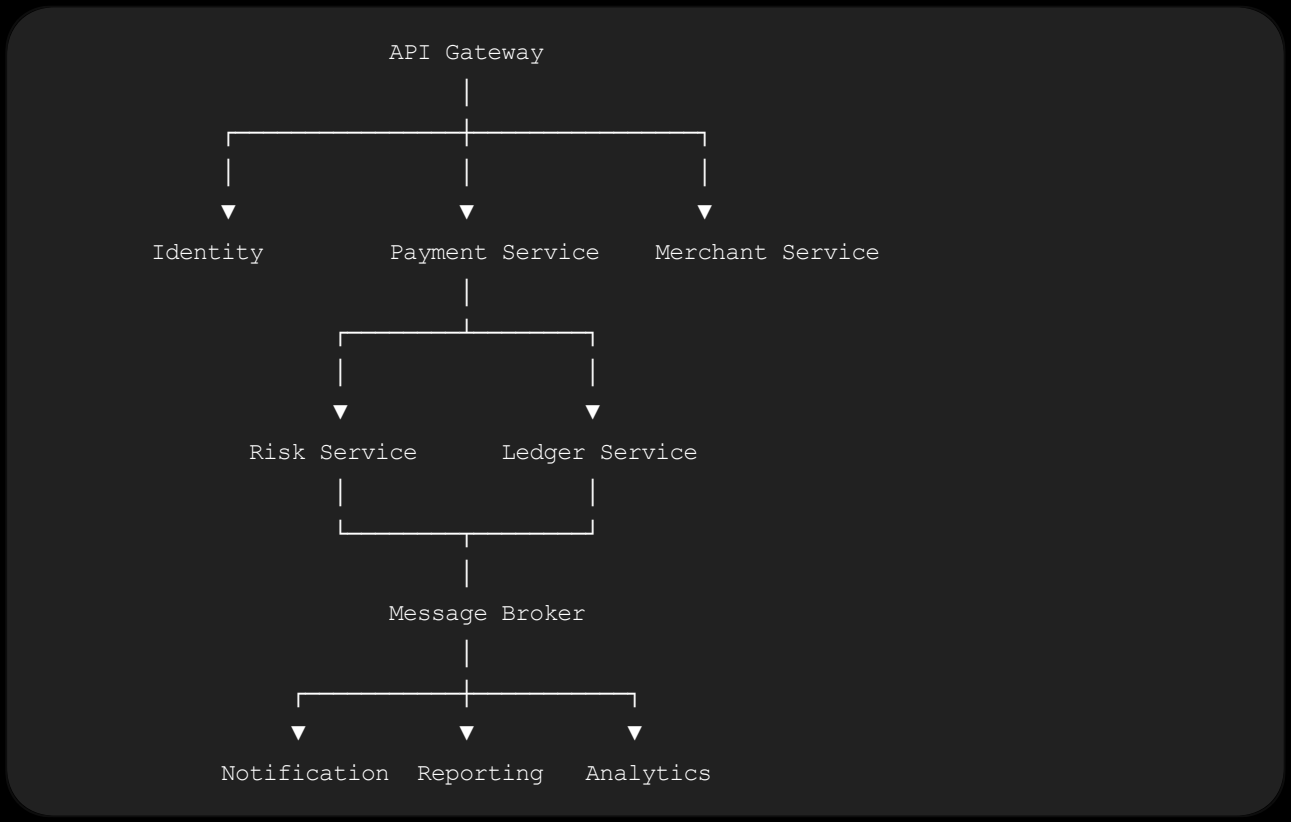
And you'll understand **why microservices are not automatically better.**

---

11. Stage 4: Microservices

Now take the architecture further.

I'd eventually arrive at something like:



Now each service can have its own database:

```
Identity       → identity_db
Merchant       → merchant_db
Payment        → payment_db
Risk           → risk_db
Ledger         → ledger_db
Notification   → notification_db
Reporting      → reporting_db
```

You don't necessarily need seven databases physically for the exercise, but **logically each service should own its data**.

12. Now you'll face real distributed-system problems

For example:

```
Payment Service
       |
       ▼
Risk Service
       |
       ▼
Payment approved
       |
       ▼
Ledger Service
       |
       ▼
Notification Service
```

What if:

```
Risk = SUCCESS
Ledger = SUCCESS
Notification = FAILED
```

Should payment fail?

No.

Now you understand:

> Not every operation belongs in one database transaction.

You start learning:

- eventual consistency
- compensating actions
- Saga pattern
- retries
- idempotent consumers
- dead-letter queues

That's **high-level architecture practice**.

---

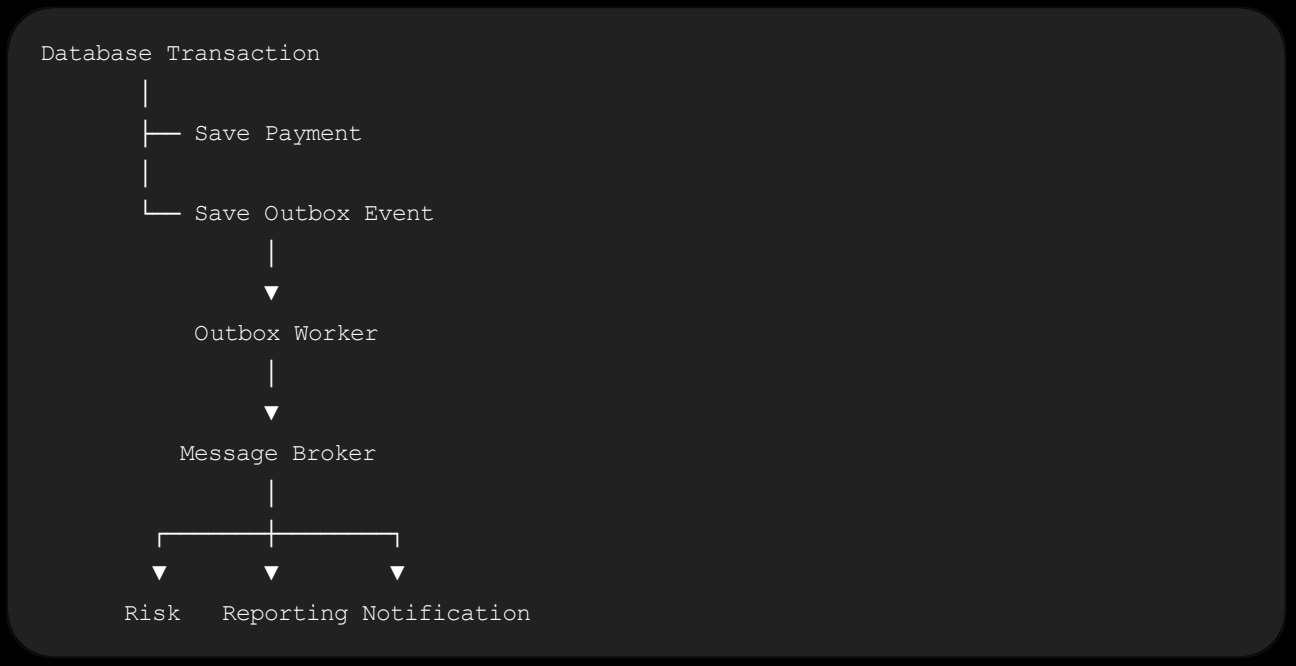
13. Add the Outbox Pattern

This is one thing I strongly recommend you implement.

Instead of:

```
Save Payment
      ↓
Publish Event
```

do:



This teaches you an extremely common real-world distributed-system pattern.

---

14. Add idempotency

For payment APIs:

```
POST /payments

Idempotency-Key:
abc-123
```

If the client sends the request twice:

```
Request 1 → Payment created
Request 2 → Same result
```

Not:

```
Request 1 → $100 payment
```

This is excellent backend practice.

---

15. Add concurrency

You should deliberately create a scenario such as:

```
Account balance = $100

Request A → withdraw $80
Request B → withdraw $80
```

Without proper concurrency control:

```
A sees $100
B sees $100

A → $20
B → $20

Final result = WRONG
```

Practice:

- database locking
- optimistic locking
- transactions
- isolation levels

This alone makes the project much more valuable than CRUD.

---

16. Add failure simulation

This is something I'd **definitely** include.

Create an admin/debug endpoint:

```
/risk/fail
/risk/slow
/payment/fail
/notification/fail
```

Then test:

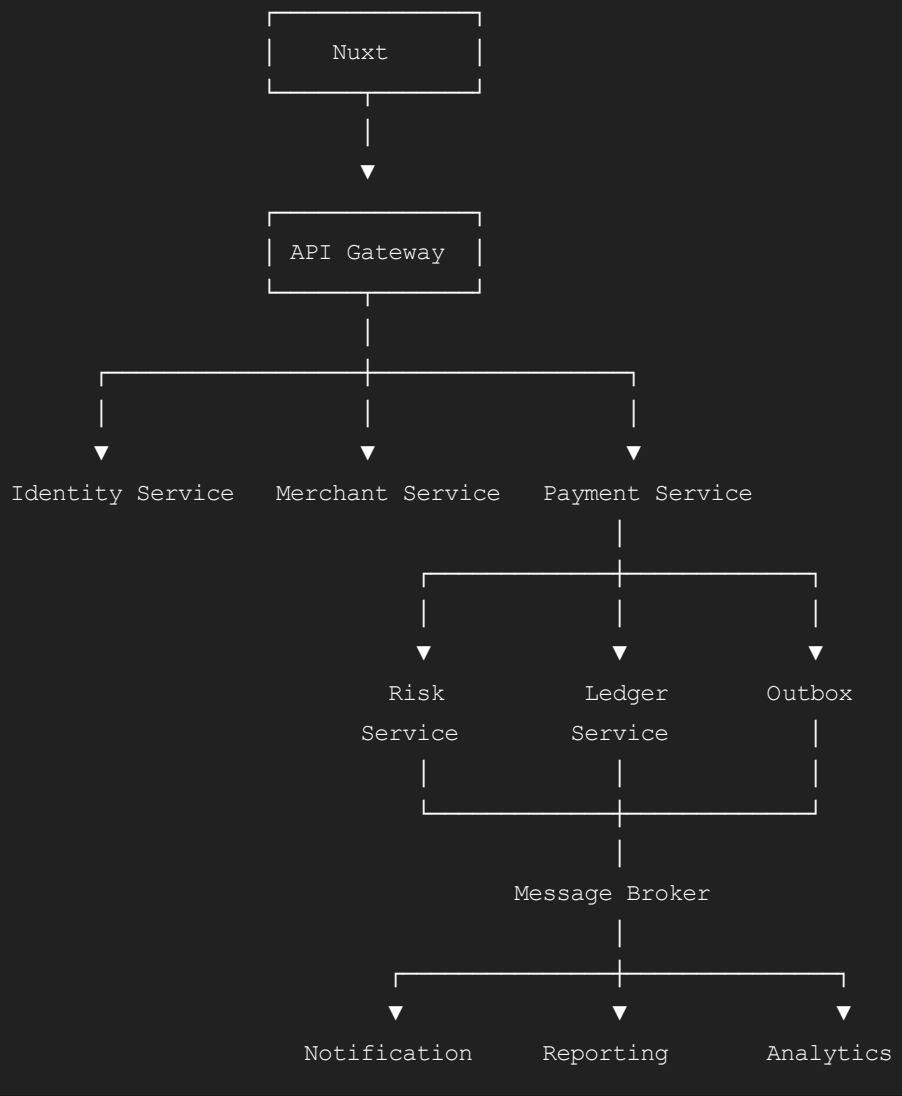
```
Risk unavailable
     ↓
Payment request
     ↓
Timeout
     ↓
Retry
     ↓
Circuit opens
```

Now you're actually practicing distributed architecture rather than just drawing boxes.

---

17. Your final project architecture

By the end, I'd want you to have something approximately like this:



That is a **very good architecture playground**.

---

18. Technology stack I'd use

Since you already work with Laravel/Nuxt, don't waste your one week learning another primary stack.

Frontend

```
Nuxt
```

Initial backend

```
Laravel
PostgreSQL
Redis
Laravel Queue
```

Hybrid / Microservices

```
Laravel
Python/FastAPI
```

Use Python particularly for:

```
Risk Service
```

That gives you practice with polyglot architecture.

Messaging

For the exercise:

```
RabbitMQ
```

or

```
Redis Streams
```

I'd lean toward **RabbitMQ** for this particular learning project because you'll get more exposure to queues, routing, acknowledgements, retries and dead-lettering.

Infrastructure

```
Docker
Docker Compose
```

You do **not** need Kubernetes in this one-week project.

That's important.

Don't turn your architecture exercise into a Kubernetes exercise.

---

19. Your one-week schedule

I'd structure your week like this.

Day 1 - Business + Monolith

Build:

```
Auth
Merchant
Customer
Payment
Risk
Ledger
Notification
```

Implement:

- authentication
- payment creation
- payment processing
- ledger
- basic dashboard

Architecture:

```
Laravel
   +
PostgreSQL
```

Day 2 - Make it a Modular Monolith

Refactor:

```
Payment Module
Risk Module
Ledger Module
```

```
Notification Module
```

Implement:

- module boundaries
- service interfaces
- domain events
- queues
- policies

Understand:

```
Coupling
Cohesion
Boundaries
Dependencies
```

Day 3 - Advanced Monolith

Implement:

- idempotency
- database transactions
- locking
- caching
- background jobs
- retries
- event-driven processing
- outbox pattern

Now your modular monolith should actually be **good**.

Day 4 - Hybrid

Extract:

```
Risk
```

into:

```
Python/FastAPI
```

Architecture:

```
Laravel
   |
   └──── HTTP ──────► Python Risk Service
```

Then extract:

```
Notification
```

using asynchronous messaging.

Now you have:

```
Modular Monolith
+
2 Services
```

That's your hybrid stage.

Day 5 - Microservices

Extract:

```
Payment
Risk
Ledger
Notification
Reporting
```

into separate services where justified.

Introduce:

```
RabbitMQ
```

and service-to-service communication.

Day 6 - Distributed Systems

This is the day I would **not skip**.

Practice:

```
Retry
Timeout
Circuit breaker
Idempotency
Dead letter queue
Outbox
Eventual consistency
Saga
Failure recovery
```

Simulate service failures.

Day 7 - Architecture + Documentation

Create:

```
Architecture v1
Architecture v2
Architecture v3
Architecture v4
```

Document:

```
Why Monolith?
       ↓
Why Modular?
       ↓
Why Hybrid?
       ↓
Why Microservices?
```

Also document:

```
What got better?
What got worse?
What complexity was introduced?
What would I choose in production?
```

This last part is extremely valuable for interviews.

---

20. One thing I DON'T want you to do

Don't do this:

```
Day 1
Build monolith

Day 2
Move files around

Day 3
Create Docker containers

Day 4
Call APIs

Day 5
Congratulations: microservices!
```

That won't teach you architecture.

Instead, at each stage ask:

> **What problem am I solving by changing the architecture?**

For example:

```
MONOLITH
    ↓
Problem: modules tightly coupled
    ↓
MODULAR MONOLITH
    ↓
Problem: Risk needs independent scaling/technology
    ↓
HYBRID
    ↓
Problem: multiple domains need independent deployment
    ↓
MICROSERVICES
```

That's the learning journey.

---

21. And here's where I disagree with your original idea slightly

You said:

> "first create a monolithic then convert that into modular monolithic and after that hybrid and at the last microservices."

Yes, do this for the learning exercise.

But don't conclude:

> "Therefore production projects should always follow this path."

No.

For a real MVP, I would probably start:

```
Modular Monolith
```

For an existing large system:

```
Existing Monolith
       ↓
Strangler Pattern
       ↓
Selective Services
```

For a system with genuinely independent domains and teams:

```
Microservices
```

Your project is different because **the objective is to learn the tradeoffs**.

---

22. What you'll be able to say after completing it

This is the real reason I like this project for you.

Instead of saying in an interview:

> "I know microservices, Docker, RabbitMQ and event-driven architecture."

You can say something much stronger:

> "I built the same payment platform through four architectural stages. I initially implemented it as a monolith, then introduced domain boundaries and converted it into a modular monolith. I later extracted the risk engine and notification processing into independent services, creating a hybrid architecture. Finally, I separated the major domains into independently deployable services and introduced asynchronous communication, an outbox pattern, idempotency, retries and eventual consistency."

That's a **much more credible senior-level answer**.

My final recommendation

For **your one-week architecture practice**, I would rank the options like this:

Approach	My recommendation
Simple e-commerce	❌
Simple CRUD SaaS	❌
Start modular → microservices	🟡 Good, but misses some learning
Build separate projects for each architecture	🟡 Too much duplication
One project: Monolith → Modular → Hybrid → Microservices	🟢 **Best for your goal**

And the project I'd choose is:

> **FinPay: a simulated payment, risk, ledger and transaction processing platform.**

It is complex enough to take you from **mid-level backend development into senior-level architecture and distributed-system concepts**, while still being possible to build a focused version in a week.

One more important point: **don't try to make the final microservices system production-complete.** Your goal is to touch the important concepts and understand their tradeoffs, not build a miniature Stripe in seven days.

If you want, I can next give you the **complete FinPay project specification**, including modules, database tables, APIs, folder structure, events, queues, architecture for all four stages, and a strict **7-day implementation checklist** so you can start coding immediately.

Give me the complete FinPay project specification

Absolutely. I would make this specification **strict enough that you don't accidentally turn it into a normal CRUD project**, but also constrained enough that you can finish the core version in one week.

The goal is not to build a production Stripe clone. The goal is to build a **serious architectural laboratory** where the same system evolves through:

Monolith → Modular Monolith → Hybrid → Microservices

FinPay: Complete Practice Project Specification

1. Project Overview

Project name

FinPay

Type

Simulated payment processing and financial operations platform.

Main objective

FinPay allows businesses/merchants to:

1. Register and authenticate
2. Create customers
3. Create payment requests
4. Process simulated payments
5. Run risk checks
6. Maintain a financial ledger
7. Receive payment notifications
8. View transaction/reporting information
9. Handle failed transactions
10. Retry failed operations

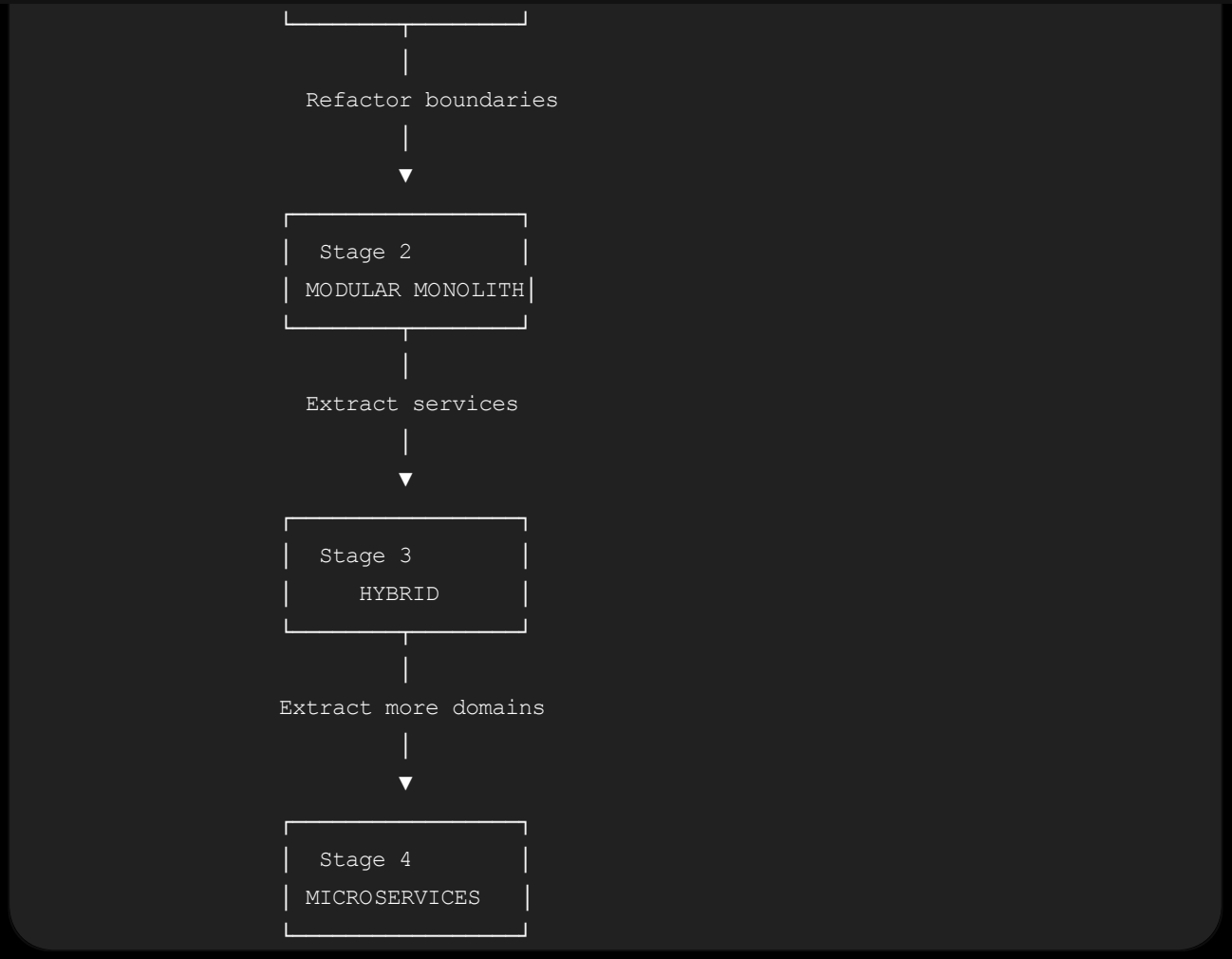
There is **no real money** and no integration with real payment providers.

---

2. Architecture Learning Objective

You will build the **same business functionality four times architecturally**.





The important thing is that **business requirements remain mostly the same**.

Only the architecture changes.

---

3. Core Business Domains

We'll use these domains:

```
FinPay
│
├── Identity
│
├── Merchant
│
├── Customer
│
├── Payment
│
├── Risk
│
├── Ledger
│
├── Notification
```

```
|
└── Reporting
```

Not every domain becomes a microservice.

That's deliberate.

You'll learn that a **domain ≠ automatically a microservice**.

---

4. Users and Roles

There are two main types of users.

Merchant Admin

Can:

- login
- manage merchant
- create customers
- create payments
- view payments
- view ledger
- view reports

Operations Admin

Can:

- view all merchants
- view all payments
- view failed payments
- inspect risk decisions
- retry failed processing
- simulate failures
- inspect system events

For the one-week project, don't build a sophisticated permission management system.

Use simple role-based authorization.

```
MERCHANT_ADMIN
OPS_ADMIN
```

---

5. Payment Lifecycle

This is the heart of the application.

A payment goes through:

```
CREATED
   ↓
RISK_CHECKING
   ↓
APPROVED
   ↓
PROCESSING
   ↓
COMPLETED
```

Possible failure:

```
CREATED
   ↓
RISK_CHECKING
   ↓
REJECTED
```

or:

```
PROCESSING
     ↓
   FAILED
     ↓
   RETRY
     ↓
 PROCESSING
```

Final statuses:

```
COMPLETED
FAILED
REJECTED
```

---

6. Payment Example

Merchant creates:

```
    http

POST /api/payments
```

JSON

```
{
    "customer_id": 101,
    "amount": 250.00,
    "currency": "USD",
    "description": "Order #10025",
    "idempotency_key": "payment-abc-123"
}
```

FinPay responds:

JSON

```
{
    "id": "pay_12345",
    "status": "RISK_CHECKING",
    "amount": 250.00,
    "currency": "USD"
}
```

Then processing occurs.

Eventually:

JSON

```
{
    "id": "pay_12345",
    "status": "COMPLETED"
}
```

---

7. Simulated Payment Processor

Don't integrate Stripe, PayPal, etc.

Create:

```
PaymentProcessor
```

with simulated behavior.

For example:

```
amount < 1000
    → success
```

```
amount >= 1000
    → 20% failure

special test card
    → failure
```

You can make this configurable.

Example:

```
CARD_SUCCESS
CARD_DECLINED
CARD_TIMEOUT
CARD_PROCESSING_ERROR
```

This lets you practice failure handling without external dependencies.

---

8. Risk Engine

The Risk domain evaluates a payment.

Example rules:

```
Amount > $5000
    → HIGH RISK

Customer has > 5 payments in 1 minute
    → HIGH RISK

Customer has previous failed payments
    → MEDIUM RISK

Normal transaction
    → LOW RISK
```

Risk result:

JSON

```
{
    "payment_id": "pay_123",
    "risk_score": 25,
    "decision": "APPROVE",
    "reasons": []
}
```

Possible decisions:

```
APPROVE
REVIEW
REJECT
```

---

9. Ledger

The ledger records financial movements.

This is where you get good database practice.

For a successful payment:

```
Payment = $100

Ledger:

Merchant Account
    +100

Customer/Processor Account
    -100
```

You don't need to model a real accounting system.

But use **double-entry concepts**.

Each transaction should have:

```
Ledger Transaction
       |
       ├── Debit Entry
       |
       └── Credit Entry
```

Example:

```
Transaction: txn_123

Debit:
    processor_account
    $100

Credit:
    merchant_account
    $100
```

The total debit must equal total credit.

```
Debit  = $100
Credit = $100

VALID
```

This gives you excellent transaction and consistency practice.

---

10. Database Design

Use PostgreSQL.

users

```
id
name
email
password
role
created_at
updated_at
```

---

merchants

```
id
name
status
api_key
created_at
updated_at
```

Statuses:

```
ACTIVE
SUSPENDED
```

---

customers

```
id
merchant_id
```

```
name
email
status
created_at
updated_at
```

---

payments

```
id
merchant_id
customer_id
amount
currency
status
risk_score
risk_decision
processor_reference
idempotency_key
failure_reason
created_at
updated_at
```

Indexes:

```
merchant_id
customer_id
status
idempotency_key
created_at
```

---

payment_attempts

```
id
payment_id
attempt_number
status
processor_reference
error_code
started_at
completed_at
```

This is useful for retry practice.

---

risk_assessments

```
id
payment_id
score
decision
reasons
created_at
```

`reasons` can initially be JSON.

---

ledger_transactions

```
id
payment_id
reference
status
created_at
```

---

ledger_entries

```
id
ledger_transaction_id
account_id
entry_type
amount
currency
created_at
```

`entry_type`:

```
DEBIT
CREDIT
```

---

accounts

```
id
merchant_id
type
balance
```

```
currency
created_at
updated_at
```

Account types:

```
MERCHANT
PROCESSOR
```

For practice, you can initially use one processor account.

---

notifications

```
id
payment_id
type
recipient
status
attempts
last_error
sent_at
created_at
```

Types:

```
PAYMENT_SUCCESS
PAYMENT_FAILED
PAYMENT_REJECTED
```

---

outbox_events

This becomes particularly important later.

```
id
aggregate_type
aggregate_id
event_type
payload
status
attempts
created_at
processed_at
```

Statuses:

```
PENDING
PROCESSED
FAILED
```

---

11. Important Database Constraints

Don't just create tables.

Implement actual constraints.

For example:

Idempotency

```
merchant_id + idempotency_key
```

must be unique.

Ledger

For each ledger transaction:

```
SUM(DEBITS) = SUM(CREDITS)
```

Payment attempts

```
payment_id + attempt_number
```

unique.

Foreign keys

Use proper relationships.

This is part of your architecture practice.

---

12. API Specification

Authentication

```
http
```

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
GET  /api/auth/me
```

Merchant APIs

```http
GET    /api/merchant
PUT    /api/merchant
GET    /api/merchant/dashboard
```

Customer APIs

```http
POST   /api/customers
GET    /api/customers
GET    /api/customers/{id}
PUT    /api/customers/{id}
DELETE /api/customers/{id}
```

CRUD is okay here because customers aren't the architectural focus.

Payment APIs

```http
POST /api/payments
GET  /api/payments
GET  /api/payments/{id}
POST /api/payments/{id}/retry
```

Optional:

```http
POST /api/payments/{id}/cancel
```

Ledger APIs

```http
GET /api/ledger
GET /api/ledger/{id}
GET /api/accounts
GET /api/accounts/{id}/balance
```

Reporting APIs

```http
GET /api/reports/payments
GET /api/reports/revenue
GET /api/reports/failures
```

Operations APIs

These are especially useful for your architecture testing.

```http
GET /api/ops/payments
GET /api/ops/failed-payments
GET /api/ops/events
GET /api/ops/health
```

---

13. Failure Simulation APIs

Create these only for development.

```http
POST /api/debug/risk/fail
POST /api/debug/risk/slow
POST /api/debug/payment/fail
POST /api/debug/payment/timeout
POST /api/debug/notification/fail
```

Or even better, create a configuration:

```
failure_mode:
    NONE
    RANDOM
    ALWAYS_FAIL
    TIMEOUT
```

Now you can test distributed failures deliberately.

---

14. Important Domain Events

Define these events.

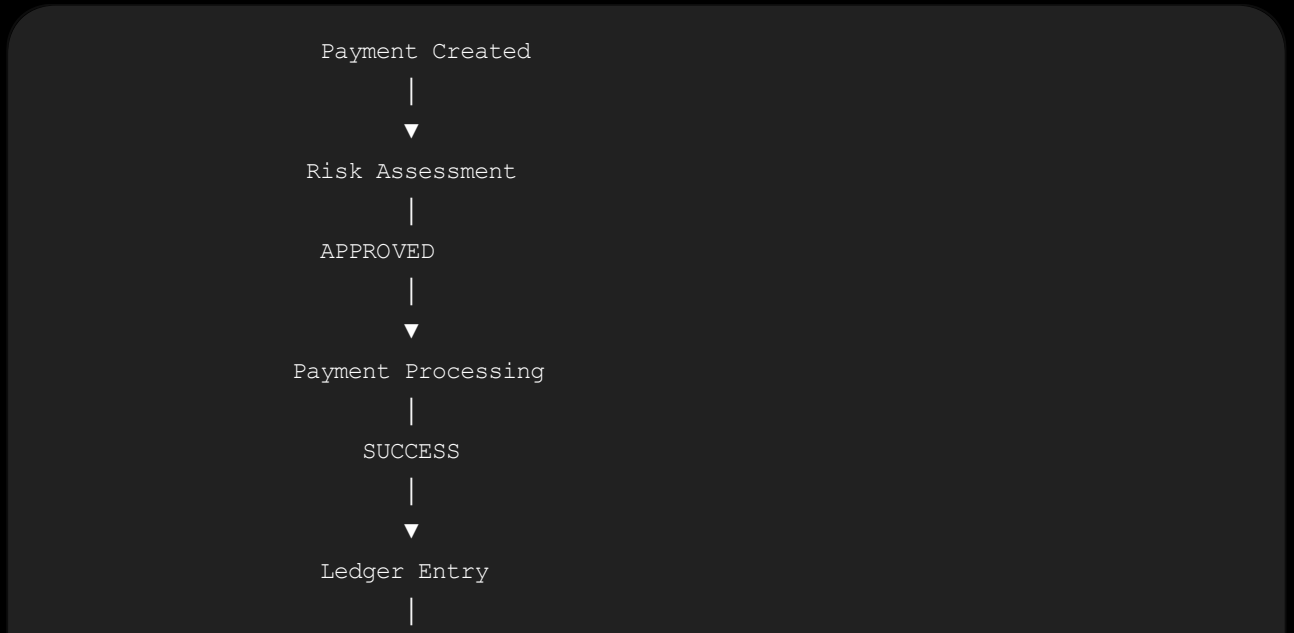
```
PaymentCreated
PaymentRiskApproved
PaymentRiskRejected
PaymentProcessingStarted
PaymentCompleted
PaymentFailed
PaymentRetryRequested
LedgerEntryCreated
NotificationRequested
NotificationSent
NotificationFailed
```

Later, these events will travel through RabbitMQ.

---

15. Event Flow

The ideal successful payment flow is:





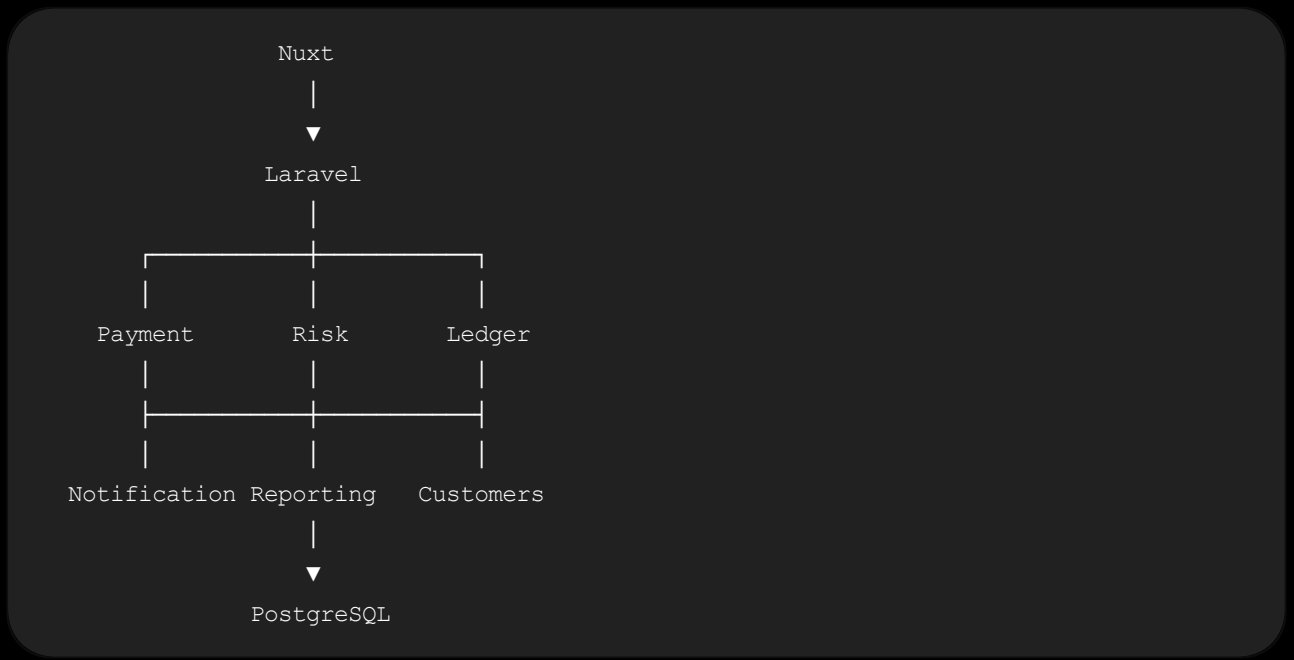
Failure:

```
Payment
   |
   ▼
Risk
   |
 REJECT
   |
   ▼
Payment Rejected
   |
   ▼
Notification
```

---

16. Stage 1 Architecture: Monolith

Everything exists in one Laravel application.



One deployment:

```
finpay-app
```

One database:

```
finpay_db
```

Redis can be used for queues/cache.

---

17. What to deliberately observe in Stage 1

Ask yourself:

Coupling

Can Payment directly access:

```
RiskAssessment model?
```

Can Reporting directly manipulate:

```
Payment model?
```

Deployment

Can you deploy Risk independently?

No.

Scaling

Can you scale Risk independently?

No.

Failure isolation

If Risk becomes unavailable, can the application isolate it?

Not easily.

Write these observations down.

---

18. Stage 2: Modular Monolith

Refactor into:

```
app/
└── Modules/
```

```
    ├── Identity/
    ├── Merchant/
    ├── Customer/
    ├── Payment/
    ├── Risk/
    ├── Ledger/
    ├── Notification/
    └── Reporting/
```

Each module:

```
Payment/
├── Application/
├── Domain/
├── Infrastructure/
└── Presentation/
```

You don't need to implement textbook-clean architecture everywhere.

The important thing is **dependency boundaries**.

---

19. Modular Monolith Dependency Rules

For example:

```
Payment
   ↓
Risk Interface
```

not:

```
Payment
   ↓
RiskController
```

And:

```
Payment
   ↓
PaymentCreated event
```

instead of:

```
Payment
   ↓
NotificationService.send()
```

This makes future extraction easier.

---

20. Introduce a Queue

Use Redis initially.

Example:

```
PaymentCreated
      |
      ▼
Queue
      |
      ▼
RiskAssessmentJob
```

Then:

```
PaymentCompleted
      |
      ▼
NotificationJob
```

Now you have:

```
HTTP request
    ↓
Business operation
    ↓
Queue
    ↓
Background worker
```

---

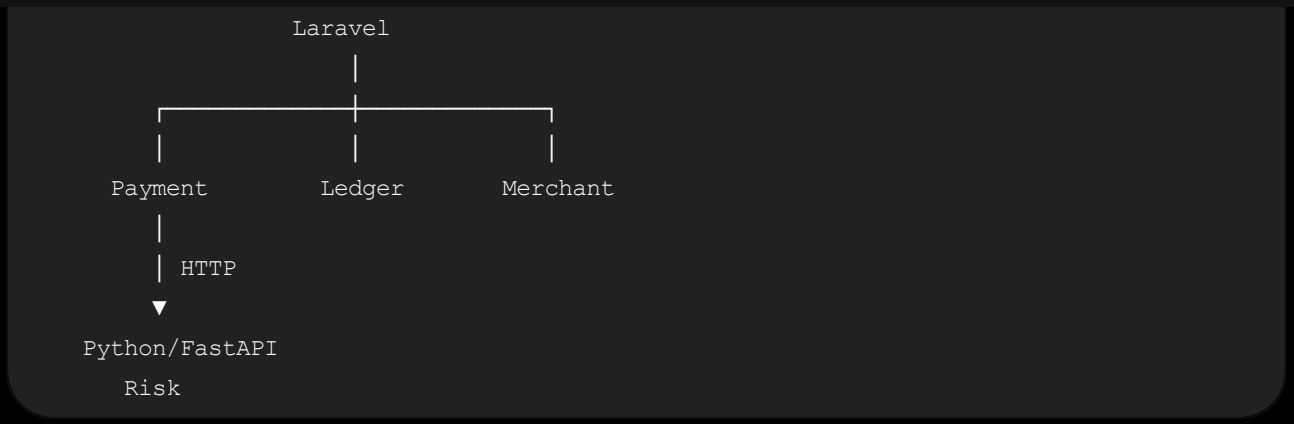
21. Stage 3: Hybrid Architecture

Now extract **Risk**.

This is intentional because it gives you a legitimate technical reason.

Risk will use Python/FastAPI.

```
                  Nuxt
                    |
                    ▼
```



Risk has:

```
POST /risk/evaluate
```

Request:

JSON

```
{
    "payment_id": "pay_123",
    "customer_id": 100,
    "amount": 500,
    "currency": "USD"
}
```

Response:

JSON

```
{
    "score": 25,
    "decision": "APPROVE",
    "reasons": []
}
```

---

22. What you'll learn here

Now deliberately break Risk.

```
Laravel
   |
   ▼
Risk
   X
```

Implement:

Timeout

```
Risk timeout = 2 seconds
```

Retry

```
attempt 1
   ↓
attempt 2
   ↓
attempt 3
```

Circuit breaker concept

```
Risk failures > threshold
        ↓
Circuit OPEN
        ↓
Don't keep hammering Risk
```

You don't need a sophisticated library if it eats your week.

Understand and implement the concept.

---

23. Extract Notification

Now use asynchronous communication.

Instead of:

```
Laravel
   |
   HTTP
   ▼
Notification Service
```

use:

```
Laravel
   |
   ▼
RabbitMQ
   |
```

For example:

```
PaymentCompleted
       |
       ▼
RabbitMQ
       |
       ▼
Notification Service
```

Now you've created your first meaningful **event-driven architecture**.

---

24. Stage 4: Microservices

Now split the system further.

I'd use:

```
services/
|
├── identity-service
├── merchant-service
├── customer-service
├── payment-service
├── risk-service
├── ledger-service
├── notification-service
└── reporting-service
```

But don't waste time making every service equally sophisticated.

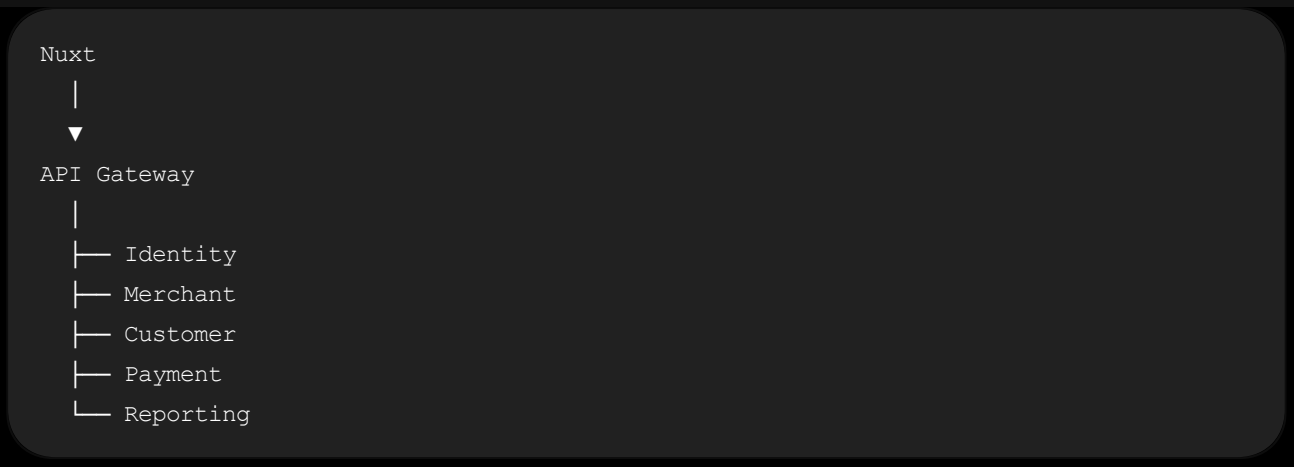
The important services are:

```
Payment
Risk
Ledger
Notification
Reporting
```

---

25. API Gateway

Introduce:



You can implement the gateway with Nginx or a small Laravel/Node gateway.

For a one-week exercise, **Nginx routing is enough**.

Don't spend two days building an API gateway.

---

26. Database-per-Service

Final architecture:

```
Identity Service
       |
 identity_db

Merchant Service
       |
 merchant_db

Payment Service
       |
 payment_db

Risk Service
       |
 risk_db

Ledger Service
       |
 ledger_db

Notification Service
       |
 notification_db

Reporting Service
       |
 reporting_db
```

This is important.

Don't create:

```
7 services
   ↓
one giant shared database
```

and call that microservices.

That's one of the easiest ways to accidentally create a **distributed monolith**.

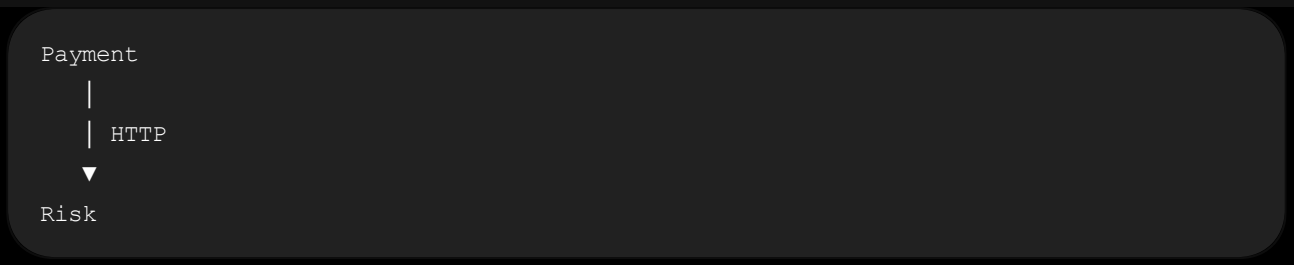
---

27. Communication Rules

Your final system should demonstrate both communication styles.

Synchronous

Use HTTP when the caller needs an immediate answer.



Example:

```
Evaluate risk
```

Asynchronous

Use RabbitMQ when immediate response isn't required.



This gives you practice deciding:

> "Should this be synchronous or asynchronous?"

That's an important architecture decision.

---

28. Outbox Pattern

This should be implemented in your modular/hybrid/microservice stages.

Bad:

```
BEGIN
Save Payment
COMMIT

Publish Event
   X
```

Now payment exists but event doesn't.

Instead:

```
BEGIN

Save Payment
Save Outbox Event

COMMIT
```

Then:

```
Outbox Worker
      |
      ▼
RabbitMQ
```

This gives you:

```
Database
    +
Message Broker
```

with much better reliability.

---

29. Idempotency

This is mandatory.

Request:

```http
POST /payments
Idempotency-Key: abc-123
```

First request:

```
abc-123
    ↓
Payment created
```

Second request:

```
abc-123
    ↓
```

```
Return existing payment
```

Not:

```
Payment 1
Payment 2
```

This becomes especially important once you have retries.

---

30. Idempotent Event Consumers

Suppose RabbitMQ delivers:

```
PaymentCompleted
```

twice.

Notification service must not send two emails.

Use an event ID:

```
event_id = evt_123
```

Consumer stores:

```
processed_events
```

or equivalent processing state.

Then:

```
evt_123
   ↓
Already processed
   ↓
Ignore
```

This is excellent distributed-system practice.

---

31. Retry Strategy

Don't simply:

```
retry 10 times immediately
```

Use exponential backoff:

```
Attempt 1 → immediately
Attempt 2 → 1 sec
Attempt 3 → 2 sec
Attempt 4 → 4 sec
Attempt 5 → 8 sec
```

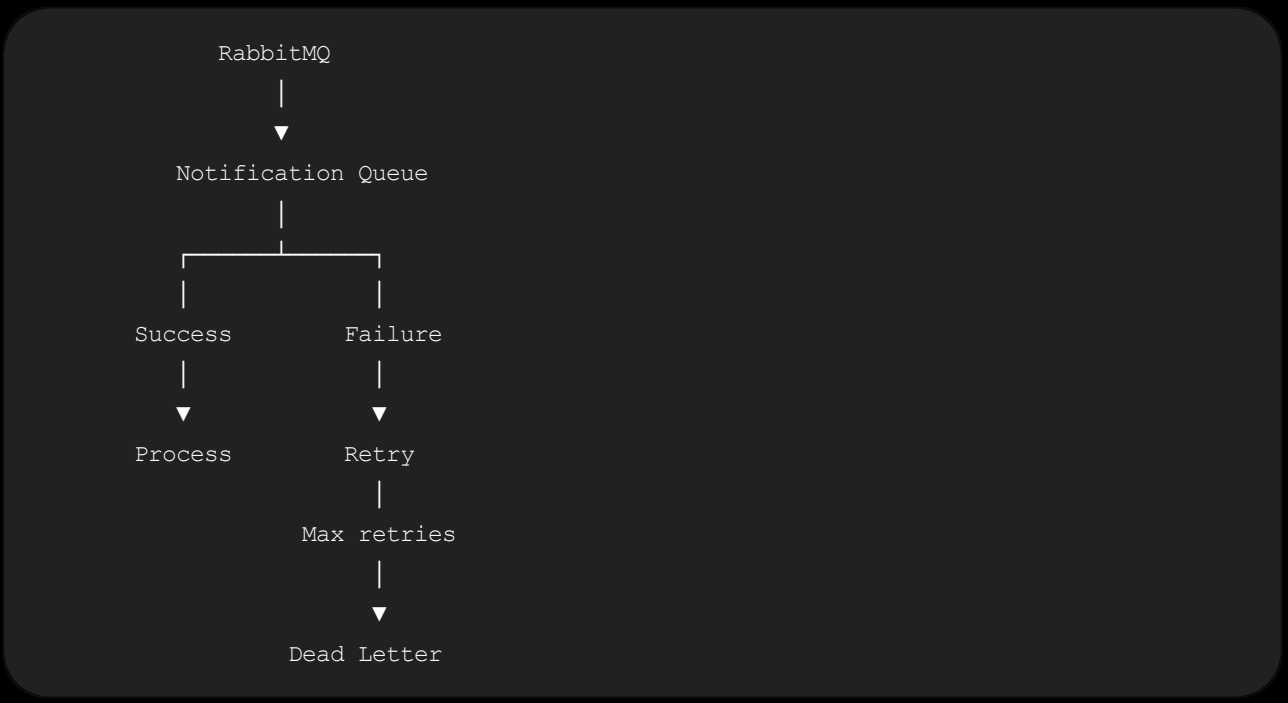
Then:

```
FAILED
   ↓
Dead Letter Queue
```

---

32. Dead Letter Queue

Architecture:



Then operations admin can inspect failed messages.

---

33. Saga Practice

Use a simplified payment saga.

```
Payment Created
      ↓
Risk Approved
      ↓
Payment Processed
      ↓
Ledger Updated
      ↓
Notification Sent
```

If ledger fails:

```
Payment Processed
      ↓
Ledger FAILED
      ↓
Compensating action
      ↓
Payment marked FAILED/REVIEW
```

Don't build a sophisticated Saga framework.

Just implement the concept.

---

34. Concurrency Practice

This is another mandatory exercise.

Suppose merchant balance:

```
$1,000
```

Two requests arrive:

```
Request A → $800
Request B → $800
```

Both cannot succeed.

Practice:

```
DB transaction
+
row locking
```

For example:

```
SELECT account
FOR UPDATE
```

Then:

```
check balance
update balance
create ledger
commit
```

This teaches you something microservices diagrams can't teach you.

---

35. Caching

Use Redis for something meaningful.

For example:

```
GET /merchant/dashboard
```

Cache:

```
dashboard:merchant:123
```

Invalidate when:

```
PaymentCompleted
PaymentFailed
```

This gives you practice with:

```
Cache-aside
Cache invalidation
TTL
```

---

36. Observability

Final system should expose:

```
/health
```

and ideally:

```
/ready
/live
```

Log:

```
request_id
correlation_id
service
event_id
payment_id
timestamp
duration
status
```

Example:

```
correlation_id: req_123

API Gateway
    ↓
Payment Service
    ↓
Risk Service
    ↓
Ledger Service
```

All logs should contain:

```
req_123
```

Then you can trace one transaction across services.

---

37. Metrics

You don't need a full Prometheus/Grafana implementation in one week.

At minimum track:

```
payment.success
payment.failed
payment.rejected
```

```
risk.success
risk.failed

notification.success
notification.failed

queue.processed
queue.failed

payment.processing_time
risk.processing_time
```

You can initially log these.

If you have time, add Prometheus/Grafana later.

---

38. Docker

Final environment:

```
docker-compose.yml
```

Something approximately like:

```
services:

  frontend
  gateway

  identity-service
  merchant-service
  payment-service
  risk-service
  ledger-service
  notification-service
  reporting-service

  postgres
  redis
  rabbitmq
```

But don't start with all of these.

Start small and grow the environment with each architecture stage.

---

39. Recommended Repository Structure

Since this is an architecture-learning project, I would actually keep the stages visible.

```
finpay/
│
├── frontend/
│
├── monolith/
│
├── modular-monolith/
│
├── services/
│   ├── payment/
│   ├── risk/
│   ├── ledger/
│   ├── notification/
│   └── reporting/
│
├── infrastructure/
│   ├── docker/
│   ├── nginx/
│   └── rabbitmq/
│
├── docs/
│   ├── architecture/
│   ├── decisions/
│   └── diagrams/
│
└── README.md
```

However, **don't maintain four independent implementations forever**.

Use Git branches/tags:

```
v1-monolith
v2-modular-monolith
v3-hybrid
v4-microservices
```

This is actually better for your learning.

---

40. Git Strategy

I recommend:

```
main
 |
 ├── tag: v1-monolith
 |
 ├── tag: v2-modular-monolith
 |
 ├── tag: v3-hybrid
 |
 └── tag: v4-microservices
```

Then your Git history itself tells the architectural story.

---

41. Architecture Decision Records

Create:

```
docs/decisions/
```

And write:

```
ADR-001 Why PostgreSQL?
ADR-002 Why Modular Monolith?
ADR-003 Why Extract Risk?
ADR-004 HTTP vs Messaging
ADR-005 Why RabbitMQ?
ADR-006 Database per Service
ADR-007 Outbox Pattern
ADR-008 Idempotency Strategy
ADR-009 Retry Strategy
ADR-010 Why Not Kubernetes?
```

This will make your project significantly stronger for interviews.

---

42. What NOT to Build

Because you have only one week, **do not build**:

```
❌ Real payment gateway integration
❌ Real bank integration
❌ KYC
❌ AML
❌ Real credit card storage
❌ Complex frontend
❌ Mobile application
❌ Kubernetes
```

```
❌ Terraform
❌ Multi-region deployment
❌ Service mesh
❌ Complex IAM
❌ Full accounting system
❌ Advanced analytics
```

Those are distractions.

---

43. What You MUST Build

Your minimum architectural feature set:

```
✓ Authentication
✓ Merchant
✓ Customers
✓ Payments
✓ Risk
✓ Ledger
✓ Notifications
✓ Reporting

✓ Transactions
✓ Concurrency control
✓ Idempotency
✓ Queues
✓ Events
✓ Retry
✓ Dead-letter queue
✓ Outbox
✓ Service-to-service communication
✓ Failure simulation
✓ Logging
✓ Correlation IDs
✓ Docker
```

That's already plenty.

---

44. Seven-Day Implementation Plan

Day 1

Goal: Working monolith

Build:

```
Auth
Merchant
Customer
Payment
Risk
Ledger
```

Implement:

```
Create Payment
       ↓
Risk
       ↓
Process
       ↓
Ledger
```

No fancy frontend.

Day 2

Goal: Strong backend

Implement:

```
Payment transactions
Idempotency
Payment attempts
Ledger double-entry
Concurrency control
```

Add:

```
Redis
Queue
Background jobs
```

Day 3

Goal: Modular Monolith

Refactor:

```
Identity
Merchant
Customer
Payment
Risk
Ledger
Notification
Reporting
```

Add:

```
Domain events
Interfaces
Module boundaries
Outbox
```

Tag:

```
v2-modular-monolith
```

Day 4

Goal: Hybrid

Extract:

```
Risk → FastAPI
```

Introduce:

```
HTTP
Timeout
Retry
Failure handling
```

Extract:

```
Notification
```

through RabbitMQ.

Architecture:

```
Laravel Modular Monolith
        |
        ├── HTTP → Risk Service
        |
        └── RabbitMQ → Notification Service
```

Tag:

```
v3-hybrid
```

Day 5

Goal: Microservices

Extract:

```
Payment
Risk
Ledger
Notification
Reporting
```

Introduce:

```
Database ownership
RabbitMQ
HTTP
Events
```

Tag:

```
v4-microservices
```

Day 6

Goal: Distributed systems

This day is extremely important.

Test:

```
Service timeout
Service unavailable
Duplicate messages
Failed messages
Retry
Dead letter queue
Concurrent requests
Database failure
RabbitMQ failure
```

Implement:

```
Idempotent consumers
Correlation IDs
Retry
Backoff
Circuit breaker concept
Outbox
Saga concept
```

Day 7

Goal: Architecture mastery

Create:

```
Architecture diagrams
ADR documents
Sequence diagrams
Failure scenarios
Performance notes
Tradeoff analysis
```

Then answer:

> Why did I choose each architecture?

---

45. The Architecture Diagrams You Should Produce

You should finish with these diagrams.

Diagram 1

Monolith

Diagram 2

Modular Monolith

Diagram 3

Hybrid

Diagram 4

Microservices

Diagram 5

Payment sequence

Diagram 6

Failure/retry flow

Diagram 7

Outbox pattern

Diagram 8

Saga/payment workflow

Diagram 9

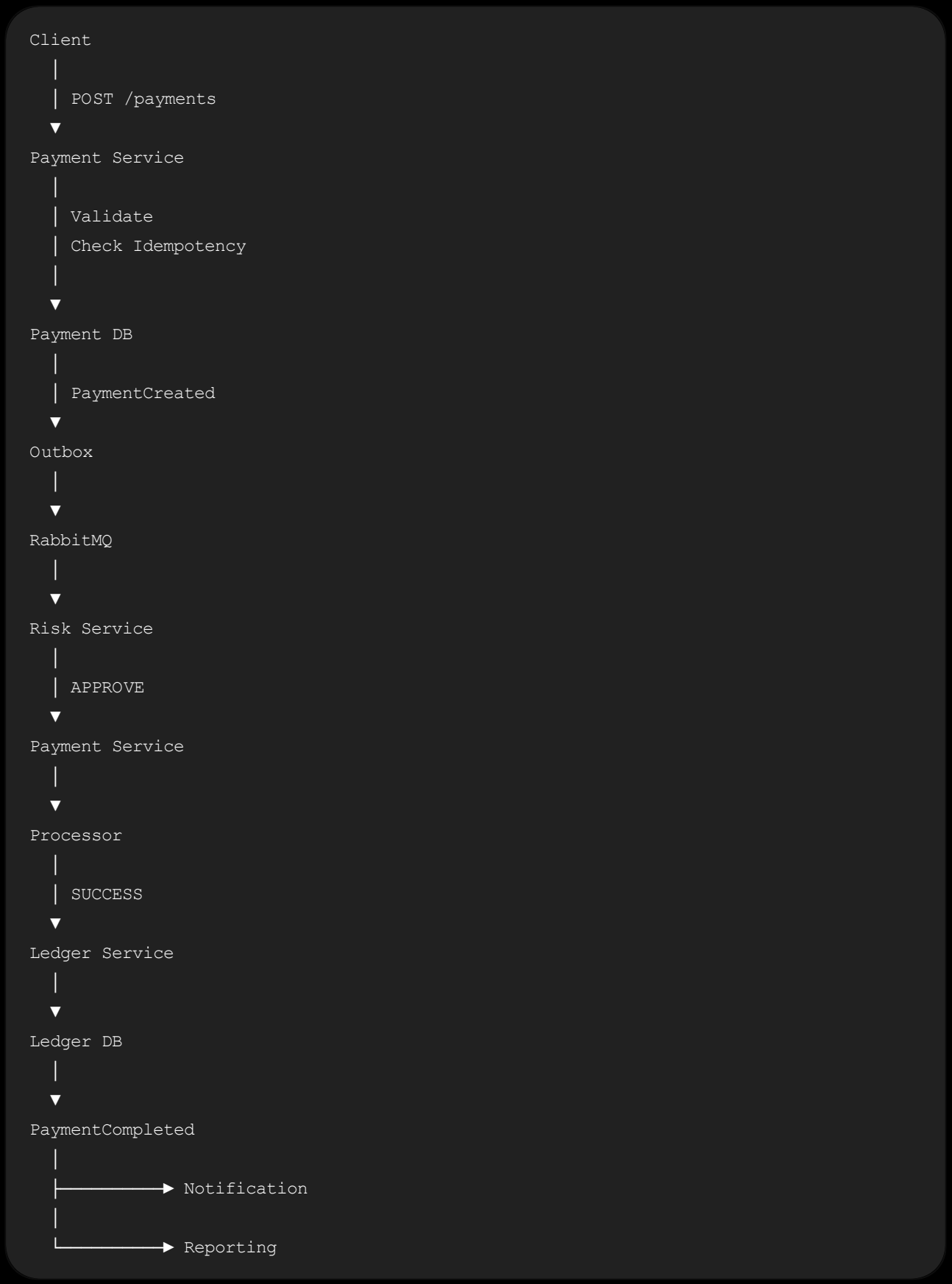
Service communication

Diagram 10

Final infrastructure

46. Your Main Payment Sequence

I want you to understand this sequence extremely well.



If you can explain this flow properly, you'll have learned a lot.

---

47. The Questions You Should Be Able to Answer

At the end, I want you to be able to answer these without memorizing textbook definitions.

Monolith

> Why didn't you use microservices initially?

Modular Monolith

> What makes your modular monolith different from a normal monolith?

Hybrid

> Why did you extract Risk?

Microservices

> Why does Payment have its own database?

Communication

> Why is Risk synchronous but Notification asynchronous?

Reliability

> What happens if Risk is unavailable?

Messaging

> What happens if RabbitMQ delivers the same message twice?

Transactions

> How do you maintain consistency between Payment and Ledger?

Outbox

> Why did you introduce the Outbox pattern?

Saga

> What happens if Ledger fails after Payment succeeds?

Scaling

> Which service would you scale first and why?

Observability

> How would you trace one payment across five services?

Architecture

> If you were building FinPay for a real client, would you choose microservices?

And importantly:

> **What would make you choose the modular monolith instead?**

If you can answer that last question honestly, you've understood the point of this exercise.

---

48. The Final Comparison You'll Make

At the end, create this table yourself based on your actual experience:

Concern	Monolith	Modular Monolith	Hybrid	Microservices
Development speed				
Deployment				
Scaling				
Fault isolation				
Data consistency				
Operational complexity				
Debugging				
Team independence				
Technology freedom				

Concern	Monolith	Modular Monolith	Hybrid	Microservices
Infrastructure cost				
Testing complexity				
Failure handling				

Don't fill this table before building the project.

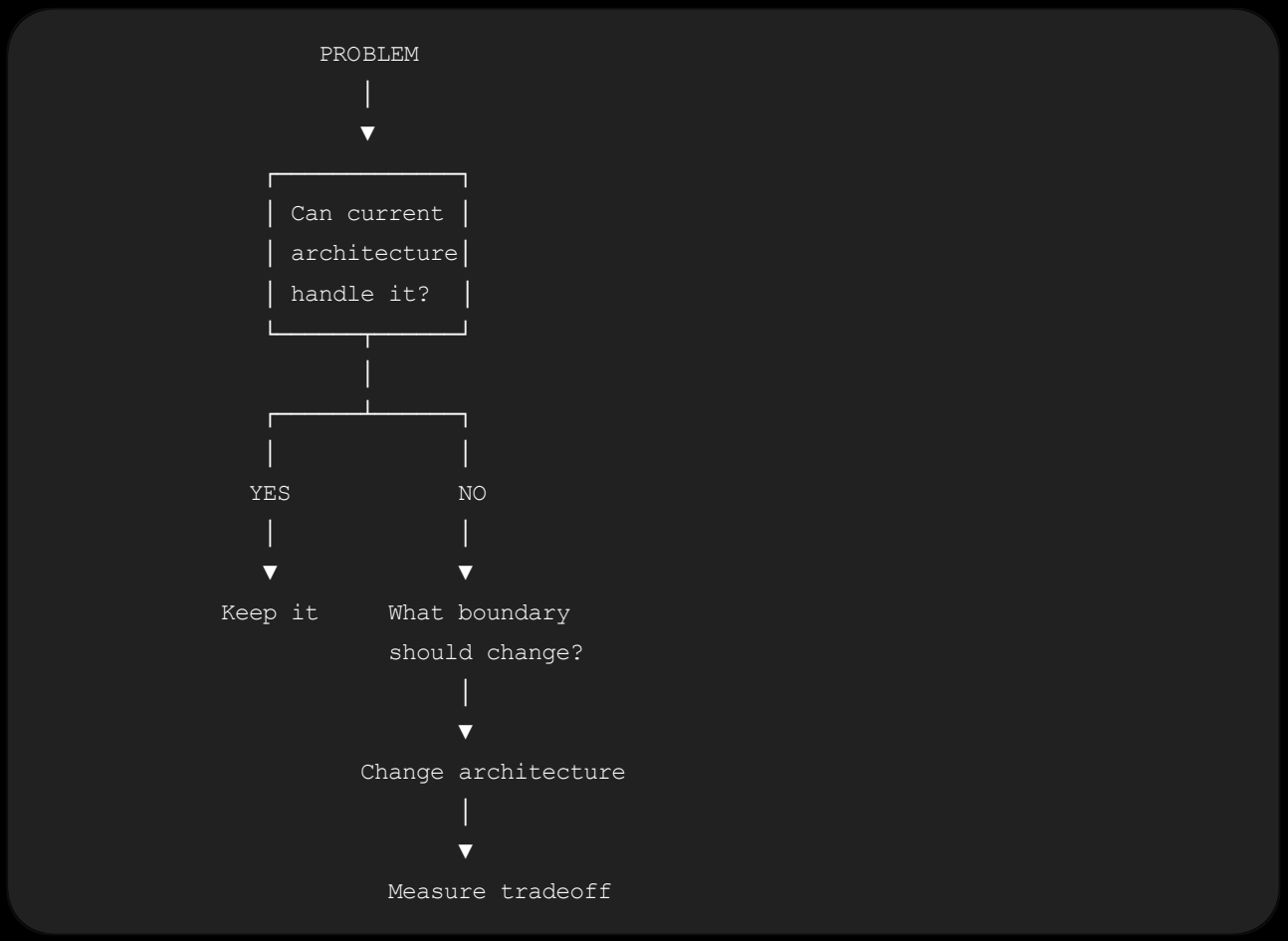
Fill it after each stage.

That's where a lot of your architectural learning will happen.

---

49. The most important learning exercise

At every stage, deliberately ask:



Don't introduce architecture because a YouTube video told you to.

Introduce it because you've encountered a problem that makes the tradeoff worthwhile.

---

50. My recommended final technology stack

Given your existing experience, I would use:

```
Frontend
————————
Nuxt


Backend
———————
Laravel


AI / Risk Service
—————————————
Python
FastAPI


Databases
—————————
PostgreSQL


Cache
—————
Redis


Messaging
—————————
RabbitMQ


Containers
——————————
Docker
Docker Compose


API
———
REST


Authentication
——————————————
Laravel Sanctum initially
```

```
Testing
_______
PHPUnit / Pest
Pytest


Documentation
_____________
Markdown
Mermaid
```

And **no Kubernetes during the one-week version**.

---

51. One final change I'd make to your original plan

You originally wanted:

> Monolith → Modular Monolith → Hybrid → Microservices

Keep that plan.

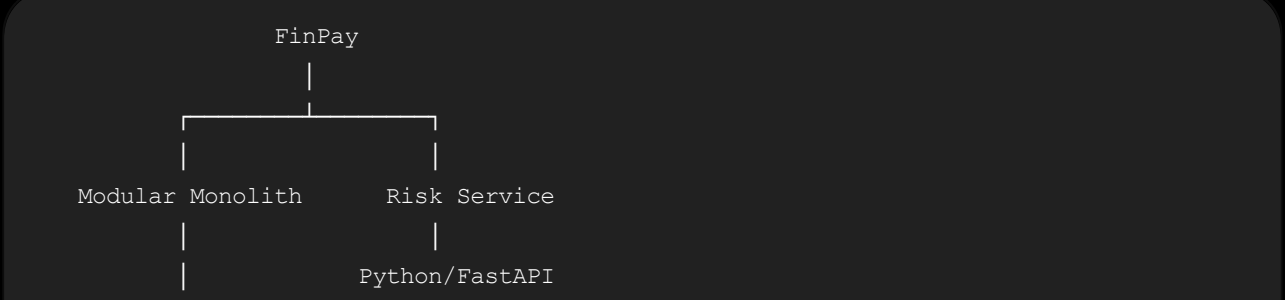
But don't think of Stage 4 as:

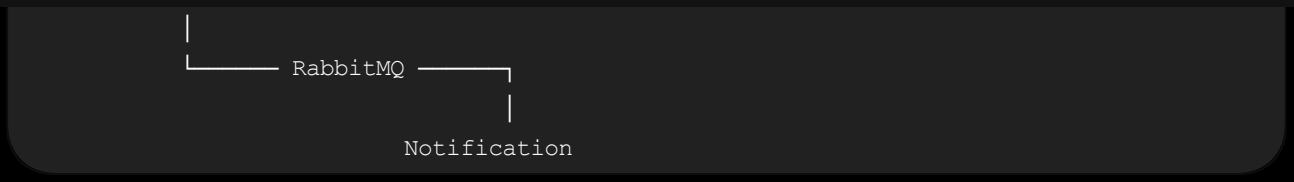
> "I must turn everything into microservices."

Instead:

```
Monolith
    ↓
Modular Monolith
    ↓
Identify real pressure points
    ↓
Hybrid
    ↓
Identify additional justified boundaries
    ↓
Microservices
```

And you may discover at the end that something like this is actually better:





That is a **valid architecture**.

You don't get extra points for having 15 microservices.

In fact, if after doing the exercise you conclude:

> "I would deploy FinPay as a modular monolith plus two independently scalable services rather than seven microservices."

That would be a sign that you learned the lesson.

The goal of this project is not to prove that microservices are superior. It is to make you capable of looking at a new project and confidently deciding **which architecture is appropriate and why**.

Today 11:12 PM

okay, good, you gave a detailed answer.
I want to use node in it rather than laravel and nuxt. If I need to build frontend then I will use react for now. let me know if I need to add frontens(react) or not. Just backend only?

Give me above reply in pdf or do so that I can follow that. if you need to make make change according to node then make it.

Yes. For **this particular practice project**, I recommend **backend-first with Node.js + TypeScript**.

My recommendation on React

Don't build React initially.

Your main objective is architecture, so spend the week on:

Node.js → PostgreSQL → Redis → RabbitMQ → Docker → distributed systems

Use Postman/Bruno and automated tests to exercise the APIs.

If you finish the architecture work early, then add a **very small React dashboard** on Day 7. Don't let frontend development reduce the time available for architecture.

I also changed the original specification from Laravel/Nuxt to:

- **Node.js + TypeScript**
- **Fastify** recommended for the API
- **PostgreSQL**
- **Prisma**
- **Redis**
- **RabbitMQ**
- **Docker + Docker Compose**
- **React only as optional frontend**
- Risk service can initially also be Node.js/TypeScript. You can experiment with Python later if you want polyglot experience.

I have prepared the complete updated specification as a PDF, including the **four architecture stages, database design, APIs, events, Outbox, idempotency, retries, DLQ, Saga, concurrency, failure simulation, Docker setup, and 7-day plan**.

Download the FinPay Architecture Practice Project Specification

My strongest advice: **don't just implement the four architectures. After each stage, document what problem forced you to change the architecture and what new complexity the change introduced.** That is what will turn this from a coding exercise into genuine architecture practice.

I can also create an image explaining the logical flow.

FinPay_Architecture_Practice_Project_Specification.pdf
PDF

I forget to mention 1 thing. Should not I use non sql like mongodb with node for SQL is better than mongo?

Yes, MongoDB is a valid option with Node.js, but **for FinPay I would deliberately choose PostgreSQL over MongoDB**.

And this is actually an important architectural learning opportunity.

Why PostgreSQL is better for FinPay

FinPay has several things that naturally benefit from relational databases:

- Payments
- Accounts
- Ledger entries
- Double-entry accounting
- Transactions
- Foreign-key relationships
- Idempotency constraints
- Concurrent balance updates
- Strong consistency
- Multi-step database transactions

For example, your ledger requires:

```
Ledger Transaction
        |
        ├── Debit Entry
        |
        └── Credit Entry

Debit = Credit
```

And you may need operations like:

```
BEGIN TRANSACTION

Lock account
Check balance
Update balance
Create ledger transaction
Create ledger entries

COMMIT
```

PostgreSQL is an excellent fit for this.

But don't interpret this as "SQL is always better than MongoDB"

That's not the lesson.

I'd actually use **FinPay to learn why you choose SQL**, rather than choosing MongoDB simply because you're using Node.js.

A better architectural question is:

> **What characteristics does my data and consistency model require?**

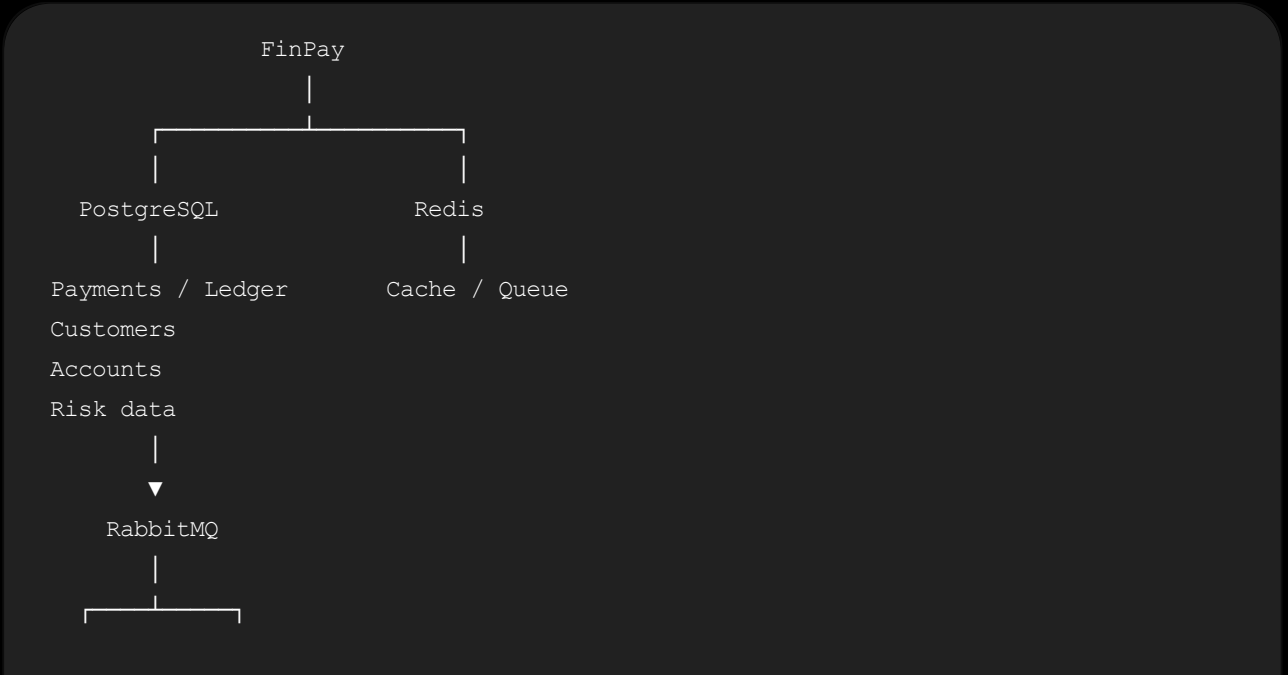
For FinPay:

Requirement	PostgreSQL	MongoDB
Payment relationships	Excellent	Good
Financial ledger	Excellent	Possible
ACID transactions	Excellent	Supported
Foreign keys	Native	No traditional FK
Unique constraints	Excellent	Supported
Complex reporting	Excellent	Good
Flexible documents	Good	Excellent
Event/log documents	Good	Excellent
Financial consistency	Excellent fit	Possible, but less natural
Learning distributed architecture	Excellent	Excellent

So **MongoDB isn't wrong**. It is just not my first choice for the core FinPay database.

What I would do for your practice project

I would actually make the project slightly more interesting:



```
▼          ▼
```

Then, **later**, if you want additional database practice, introduce MongoDB for something that genuinely fits it, such as:

```
Analytics / Audit / Event History
              |
              ▼
           MongoDB
```

For example, storing an immutable-ish event/audit document:

```json
{
  "event": "PaymentCompleted",
  "paymentId": "pay_123",
  "merchantId": "merchant_45",
  "timestamp": "...",
  "metadata": {
    "riskScore": 21,
    "processor": "simulated"
  }
}
```

That would teach you something much more valuable than simply replacing PostgreSQL with MongoDB.

One more important point

Since your goal is **middle-level → high-level architecture**, I would **not change the project to MongoDB just to practice NoSQL**.

You already have a lot to learn:

Monolith
→ Modular Monolith
→ Hybrid
→ Microservices
→ HTTP
→ RabbitMQ
→ Outbox
→ Idempotency
→ Retries
→ DLQ
→ Concurrency

→ **Transactions**
→ **Eventual consistency**
→ **Saga**
→ **Observability**

Adding MongoDB to the first week can become another technology rabbit hole.

My final recommendation

Use:

```
Node.js
TypeScript
Fastify
PostgreSQL
Prisma
Redis
RabbitMQ
Docker
```

No React initially.

Then after you complete FinPay, I would recommend a **second smaller exercise specifically for MongoDB + Node.js**, where MongoDB is actually the better architectural choice.

That will give you a much stronger understanding of **when to use SQL vs NoSQL**, instead of just learning how to use MongoDB.

And yes, **I would keep PostgreSQL in the PDF specification I gave you.**

ChatGPT can make mistakes. Check important info.